

在线二部匹配中预测的局限性：一项统一实验研究

[作者姓名] ——硕士学位论文（草稿）

执行摘要

在线二部匹配是不可撤销顺序分配的标准模型：资源事先已知，请求逐个到达，每个请求都必须在看到其余输入之前，被匹配到一个空闲且兼容的资源，或被丢弃。它是在线广告、网约车派单与请求路由背后的抽象。近年一条研究路线为这类算法提供关于输入的**预测**，并要求预测准确时性能接近最优（**一致性**）、预测错误时不出现损失（**鲁棒性**）。

针对匹配问题存在两族这样的算法，但此前各自被孤立分析，且几乎全部停留在最坏情况理论，因此预测究竟值多少并不清楚。本项目把它们放到同一套评测框架上（相同的图、到达序列、最优解与配对随机种子，四种结构化误差模型，全程 95% 置信区间），先复现一项已发表的实验研究做校验，再在合成图族、六个真实世界图与真实 trace 上运行。各种设定下结论一致：免预测基线本已接近最优（离线最优的 0.990），留给好预测的余地不足 0.01，而无防护地跟随预测会崩到 0.472。此处预测是**鲁棒性保险**，而非**性能杠杆**：本文研究的是预测的局限，而不是一个更好的算法。

- 我搭建了实验框架，并在加入任何与预测相关的工作之前，用 Borodin、Karavasilis 与 Pankratov 已发表的研究对其做了校验（第 8–12 页）。
- 我做出了两族算法的首个统一基准：按原论文实现了 Aamand–Chen–Indyk、Choo 等人与 Burathep–Erlebach–Moses 的算法，并统一到共同的误差模型与最优解上；其中每一处显著差距都出现在**下行**一侧（第 13–15 页）。
- 我刻画了残余损失由什么支配：Kendall- τ 顺序误差（Spearman $\rho = 0.979$ ），而已知的 n – LIS 界宽松 16–75 倍；顺序依赖性本身是 Aamand–Chen–Indyk 的结论（第 16–17 页）。
- 我完成了对测试-回退机制的首个实证研究（一种更准确的检验反而给出更差决策的病态，以及一个分辨率极限；第 18–21 页），并在六个真实图与一个廉价的非机器学习预测器上验证了外部效率（第 22–25 页）。
- 我报告了两个负结果（第 26–32 页），并证明了一条一致性/鲁棒性权衡不等式，据此在基准参数下不可捕获的上升空间约为 0.004，恰好是实测到的量级（第 33–37 页）。

目录

第一章 引言	1
第二章 背景与相关工作	4
第三章 模型、算法与方法	8
第四章 统一基准	13
第五章 什么支配损失：顺序误差，以及已知的界没有说的事	16
第六章 深入研究测试-回退	18
第七章 外部效度	22
第八章 探索方向与负结果	26
第九章 应用案例研究：AI 推理服务	30
第十章 结论与未来工作	33
参考文献	38
附录 A 复现指南	40

第一章 引言

1.1 研究背景与动机

在线二部匹配是不可逆序贯分配的经典模型。离线资源事先已知；请求逐个到达；每个请求必须在看到其余输入之前，立即、不可撤销地被匹配到一个可用的相容资源，或被丢弃。它是在线广告投放、网约车派单、器官交换以及现代服务系统中请求路由背后的抽象。由于决策无法撤销，全部难度都在于**在对未来不确定的情况下做出良好的即时承诺**。

近年一个有影响力的思路是：给这样的算法一个关于输入的**预测**（哪些资源会被争用，或每类请求将到达多少），并要求两个保证同时成立：**一致性 (consistency)**，即预测准确时接近最优；以及**鲁棒性 (robustness)**，即预测错误时不差于一个免预测算法。（全文一律称这个对象为**预测**，只有在讨论“是否信任它”时才称**建议**；二者指同一样东西。）针对在线匹配，具体出现了两族这样的**学习增强 (learning-augmented)** 算法：MinPredictedDegree，它消费的是对每个资源度数的预测，即有多少种请求会争抢它，也就是它会被争用到什么程度；以及一族**测试-回退 (test-and-fallback)** 方案，它们在一段较短的到达前缀上检验请求类型预测，再决定是否信任它。

这些算法都带有诱人的最坏情况保证，然而正如本文将以实验证明的，它们在平均情况下的行为却出奇地平淡：在真实负载上，这些成熟的预测很少真的**带来帮助**，而算法的真正价值在于它们不会**造成损害**。这种“最坏情况承诺”与“平均情况现实”之间的落差，正是本文的出发点。

此外，这些算法此前都只被**孤立地**研究：各自在自己的输入模型上、针对自己的预测误差概念，且主要通过理论。因此没有一个共同的基准来比较它们、量化预测究竟带来多少收益、或解释这种落差。本文提供这一共同基准，并最终给出一个解释。

1.2 研究目标

全文的性能度量统一为：算法所得匹配与同一实例上离线最优之比（3.1 节给出精确定义）。本文围绕三个问题展开：

1. 这些学习增强的在线匹配算法在同一预测误差模型、同一输入下正面比较，究竟表现如何：在真实数据上，预测帮助多少、代价几何？

2. 自适应的测试-回退机制的失效模式是什么：其检验的代价、其接受/拒绝决策的校准、以及它在何处出错？
3. 为什么平均情况的体验与最坏情况的承诺差别如此之大：观察到的这堵墙是特定算法与生成器的产物，还是必然的？

前两个问题通过实验回答；第三个通过理论回答。

1.3 主要贡献

- 一个统一实验基准（第 4 章；对应问题 1）：这些族的首次正面比较，置于同一套实验框架之上，即单一代码路径，其中每个算法看到相同的图、相同的预测与相同的最优解，配以**结构化**误差模型（误差沿实例的结构注入，而非独立同分布的噪声）与置信区间。
- 把预测刻画为鲁棒性保险（第 4、7 章；对应问题 1）：无防护地跟随预测会崩到**低于**免预测基线；结构式与自适应式两种机制以不同取舍恢复安全性；而一致性上升空间很小，因此价值在于下行保护。该图景在合成图、六个真实世界图与真实 trace 上得到验证，包括用一个廉价、非机器学习的历史统计预测器。
- 对顺序误差理论的实证性对接（第 5 章；对应问题 1）：MinPredictedDegree 的损失由一个 Kendall- τ 顺序误差支配（即被预测排错相对次序的资源对所占的比例），四种误差模型塌缩到它之上。顺序依赖本身是 Aamand-Chen-Indyk 的结果；本文的贡献是刻画其界的松弛与饱和，以及正确的顺序度量。
- 对测试-回退的首个实证研究（第 6 章；对应问题 2）：其检验代价、一个阈值校准的病理现象（**更准确的**检验反而做出**更差的**决策）、重校准所暴露的分辨率极限，以及一个不可逆惩罚，它解释了为何匹配需要先**测试再一次性提交**（test-then-commit）。
- 对墙为何矗立的量化解释（第 6 章；对应问题 3）。所谓墙，指的是在平均情况输入上，无论换更好的算法还是更好的预测，竞争比都不会有明显变化。我们把它追溯到一个**分辨率极限**（没有任何实用阈值能捕获低于其自身估计器噪声的上升空间），而这一极限在整个难度范围上持续成立（图 6.4）。

除上述之外，本文还记录了（第 8 章）所探索的方向及其返回的负结果：一次试图**学习**预测器以榨取更多性能的尝试，以及一次试图找到预测真正有帮助的服务场景的尝试。两者都强化了核心发现；它们提出的问题——这堵墙是否被迫？——由结论章的展望（10.2 节）接手。该展望属于讨论而非又一项贡献：它证明了一条一致性/鲁棒性权衡不等式，并据此解读本文自己的实验。除该不等式外，本文任何地方都不声称定理。

1.4 论文组织

各章按上述三个问题分组。第 2、3 章建立共同基础：背景（输入模型、学习增强范式、具体算法，以及这些算法的检验背后的分布检验结果），以及本文自己的模型、方法与实验框架，后者通过复现 Borodin 等人研究的一个子集加以验证。第 4 至 7 章回答问题 1 与问题 2：统一基准及其四个发现（第 4 章）、支配那微小损失的量（第 5 章）、测试-回退机制的深入研究（第 6 章），以及在真实预测器与真实图上的外部效度（第 7 章）。第 8 至 10 章处理问题 3 与边界：所探索的方向及其负结果（第 8 章）、AI 推理服务的应用案例研究（第 9 章），以及总结全文、给出关于墙为何矗立的理论展望（10.2 节）、对照研究目标做批判性评估（10.3 节）并陈述局限与未来工作的结论章（第 10 章）。有一句话贯穿全文，10.1 节给出其完整形式：在平均情况的在线匹配上，预测是鲁棒性保险而非性能杠杆。

第二章 背景与相关工作

本章建立全文所依赖的技术背景：在线二部匹配问题及其输入模型（2.1 节）、本文所处的 known-i.i.d. 模型（2.2 节）、学习增强（“带预测”）范式及其一致性/鲁棒性语言（2.3 节）、本文所基准和界定的具体基于预测的匹配算法（2.4 节）、算法的检验与 10.2 节展望背后的分布检验结果（2.5 节），以及本文所填补的文献空白（2.6 节）。

2.1 在线二部匹配与竞争分析

在在线二部匹配中，二部图的一侧（**离线资源**）事先已知，而另一侧的顶点（**在线请求**）逐个到达。每个请求到达时，算法看到其关联的边，必须**立即、不可撤销地**将其匹配到一个当前未匹配的邻居，或不予匹配。目标是最大化匹配的规模（在带权变体中为价值）。性能以**竞争比**衡量：算法所得匹配与同一输入上离线最优之比，期望同时对随机输入与算法自身的随机性取（3.1 节固定我们实际报告的形式）。

问题的难度完全取决于所假设的**输入模型**：

- **对抗到达序**。请求及其顺序由对手选择。Karp、Vazirani 与 Vazirani 的开创性结果 [1] 表明，随机化的 **Ranking** 算法（预先对资源固定一个均匀随机的优先序，将每个请求匹配到其优先级最高的可用邻居）可达竞争比 $1 - 1/e \approx 0.632$ ，且这在对抗模型下是最优的。确定性算法无法超过 $1/2$ 。
- **随机到达序**。图是對抗的，但请求以均匀随机的顺序到达；这严格易于对抗序，可达高于 $1 - 1/e$ 的比值。
- **Known-i.i.d.**。请求独立同分布地从一个**已知**的请求类型分布中抽取（见 2.2 节）；这是平均情况模型，也是本文所研究的模型。

这三个模型在难度上是嵌套的，而方向在全文中都很重要：每个 known-i.i.d. 实例也是一个随机序实例，每个随机序实例也是一个对抗序实例。因此在更难模型中证明的保证在更易模型中仍然成立，但反过来不成立，所以一个为对抗序证明的界，除了给出下限之外，对算法在 known-i.i.d. 输入上能做得**多好**并无说明。

2.2 Known-i.i.d. 模型

在 known-i.i.d. 模型中, 存在一个二部类型图: 每个在线类型 ℓ 在离线资源中有固定的邻域 $N(\ell)$, 一个实例由从已知分布 p 上独立抽取 m 个到达生成。类型图与 p 已知, 而实现的到达序列未知。它刻画了这样的场景 (广告投放、周期性请求流), 其中请求的**总体**在统计上稳定, 尽管单个到达并非如此。

一系列工作将 (对类型图取最坏情况的) 竞争比推高至 $1 - 1/e$ 对抗壁垒之上: Feldman 等人 [2] 首先通过对一个建议匹配做基于流的**蓝/红分解**突破了它 (0.670); Manshadi、Oveis Gharan 与 Saberi [3] 及 Jaillet-Lu [4] 用基于 LP 与基于列表的在线策略进一步改进了比值 (分别到 ≈ 0.702 与 ≈ 0.729); 后续工作继续精化。这些即是那些以最坏情况最优性为动机的“成熟”算法。

对本文至关重要, 的是, Borodin、Karavasilis 与 Pankratov [5] 对这些算法做了实验研究, 发现在 **平均情况**的 i.i.d. 实例上, 图景与最坏情况大不相同: 简单算法 (Greedy、Ranking) 的表现几乎与成熟算法一样好, 后者的优势是最坏情况而非典型情况的现象。

这条线索上的一个经验观察正是本文的种子: 在典型输入上, 简单基线本已近乎最优。此后的一切, 都是在追问预测还能在它之上添加什么; 我们复现该研究的一个子集作为验证 (第 3 章)。

2.3 学习增强算法

算法带预测 (或称**学习增强**) 范式 (综述见 [6]) 为一个在线算法配备关于未知输入的预测, 并要求同时给出两个保证:

- **一致性**: 预测准确时接近最优;
- **鲁棒性**: 预测任意错误时不差于一个免预测的保证。

该范式由 Lykouris 与 Vassilvitskii [7] 在竞争缓存中确立, 他们展示了如何在信任与忽略预测器之间插值, 同时对一致性与鲁棒性给出界。随后在滑雪租赁、调度等在线问题上出现了大量文献, 包括 Wei 与 Zhang [8] 关于**最优一致性/鲁棒性**取舍的分析, 它给出了两目标之间问题固有的 Pareto 前沿。近期工作还在分布鲁棒的表述下分析了可达比值如何随预测精度**连续退化** [9], 与本文所用的离散一致性/鲁棒性端点互补。

这套文献里有两样东西对本文是承重的。反复出现的机制是**对冲**: 将预测与一个安全默认相结合, 使坏预测无法造成灾难。Chłędowski、Polak、Szabucki 与 Żoła [10] 的盲从-带切换 **组合器** (combiner) 即是一种此类机制, 我们将其移植并做基准评测 (第 6 章); 他们那篇关于鲁棒学习增强缓存的实验研究, 也是本文实验半部最接近的方法学模板。

2.4 面向在线二部匹配的预测

两条线索将带预测范式应用于在线匹配, 区别在于它们所消费的**预测对象**。

度数预测:MinPredictedDegree。Aamand、Chen 与 Indyk [11] 提出 **MinPredictedDegree(MPD)**: 给定对每个离线资源度数（其将被争用的程度）的预测 μ ，将每个到达匹配到其**预测度数最小**的可用邻居，即保护预测最稀缺的资源。MPD 天生鲁棒：一个常数（无用）预测器使它退化为 Ranking。一个关键结构性事实（本文第 5 章将对接）是：MPD 仅通过 μ **诱导的顺序**依赖于它；作者的附录 D 用一个分两步构造的顺序量为匹配损失定界：先把真实度数按预测给出的顺序排成一列，再数其中有多少个位置不对，形式上即 $n - \text{LIS}$ ，其中 LIS 是该序列最长非降子序列的长度。当且仅当预测把顺序排对时这个计数为零，因此一个单调（保序）预测导致零损失。

类型直方图建议与测试-回退。第二条线索取预测为类型上的直方图 $\hat{c}$ （各类型将到达多少）。Choo 等人 [12] 提出 **TestAndMatch**：由 $\hat{c}$ 构造一个匹配并模仿（Mimic）它，但先花一段亚线性到达前缀**检验**观测到的类型频率是否与 $\hat{c}$ 相符（使用一个改编自 Jiao、Han 与 Weissman [13] 的 ℓ_1 -距离检验器），若不符则回退到 Ranking。Burathep、Erlebach 与 Moses [14] 将其（“Test-and-Match+”）推广到随机到达模型以及对匹配规模的不完全知识。两文均给出**上界**（算法）；其唯一的下界（Choo 等人的定理 3.1）是一个一般性的**对抗**不可区分性结果（无算法能既 1-一致又 $> \frac{1}{2}$ -鲁棒），二者都未在随机模型中证明下界。值得注意的是，Choo 等人的接受阈值已经与基线竞争比 β **耦合**，但是构造性地、在算法设计内部，而非作为一个下界。该耦合的代价几何，即跟随/回退的决策从根本上需要多长的前缀，正是本文所测量（第 6 章）并据以做定量解读（10.2 节）的问题。

2.5 分布检验

上述算法核心的检验，本质上是**分布检验**中的一个问题：给定来自未知分布 p （支撑大小为 r ，它就是本文模型中的 r ，即请求类型数，因为直方图预测正是类型上的一个分布）的样本以及一个已知参照 q ，判定 p 与 q 相距多远。本文全程以 ℓ_1 度量这个距离，与算法和实验保持一致；它是全变差距离的**两倍**，本文引用的每一个阈值都是 ℓ_1 阈值。必须区分两个体制：

- **恒等（非容差）检验**，即区分 $p = q$ 与 $\|p - q\|_1 \geq \varepsilon$ ，其样本复杂度为 $\Theta(\sqrt{r}/\varepsilon^2)$ [15, 16]，关于支撑大小是**亚线性的**。
- **容差检验/距离估计**，即区分 $\|p - q\|_1 \leq \varepsilon_1$ 与 $\geq \varepsilon_2$ ($0 < \varepsilon_1 < \varepsilon_2$)，也就是估计距离而非检验相等，可证要**难得多**：Valiant 与 Valiant [17] 表明它需要 $\Theta(r/\log r)$ 个样本，关于支撑**近乎线性**。Jiao、Han 与 Weissman [13] 给出了 ℓ_1 -距离估计的匹配界，Canonne、Jain、Kamath 与 Li [18] 精确刻画了整个 $(\varepsilon_1, \varepsilon_2)$ 图景，表明对常数容差，代价跃升到“勉强亚线性”的 $\tilde{\Theta}(r/\log r)$ 。

$\sqrt{r}$ （检验）与 $r/\log r$ （容差检验）之间近乎二次的差距在本文中出现两次。它是 TestAndMatch 的可部署版本退回经验代理的原因，也是该代理在大支撑上失明的原因，即第 6 章测得的分辨率极限。这一壁垒是否也封死其他一切决策规则，比乍看之下微妙得多；结论章的展望（10.2 节）将回到这一点。

2.6 本文的定位

在此背景下，三个空白凸显出来。它们与 1.2 节的三个问题一一对应，本文也按该顺序填补：

1. **缺乏统一的实证比较。**上述匹配算法各自被孤立地研究，在各自的输入族与误差模型上，且主要通过理论；没有一个在共同框架下的正面实验基准。（第 4-7 章。）
2. **缺乏对测试-回退的实证研究。**[12, 14] 核心的分布检验没有可部署的实现（作者自己也退回到一个经验代理），其检验代价、阈值校准与失效模式未被测量。（第 6 章。）
3. **缺乏对前缀检验代价的量化。**此前无工作测量或界定：在强基线实例（可捕获上升空间最小之处）上，跟随/回退的决策需要多长的前缀。支撑这一断言的先行研究检索见 8.3 节。（第 6 章实证；10.2 节展望。）

本文用实验填补前两个空白，并在第三个上迈出第一步量化的步伐。

第三章 模型、算法与方法

第 2 章将问题置于其领域中。本章固定全文所用的精确模型与记号 (3.1 节)，详述我们所实现的算法 (3.2 节)、预测对象与结构化误差模型 (3.3 节)、一致性/鲁棒性度量 (3.4 节)，以及实验框架 (3.5 节)。最后，在任何贡献建立于其上之前，通过复现 Borodin 等人已发表的图来验证该框架 (3.6 节)。

3.1 精确的 known-i.i.d. 模型

存在一个包含 n 个离线资源的集合 R ，以及一个含 r 个在线类型的类型图；类型 ℓ 的邻域为 $N(\ell) \subseteq R$ 。一个实例由从类型分布 p 上独立同分布地抽取 m 个到达生成；第 i 个到达揭示其类型（从而其邻域），必须立即、不可撤销地被匹配到其邻域中一个当前未匹配的资源，或不予匹配。类型图与 p 对算法已知，实现的到达序列未知。遵循标准的**整型类型**约定，默认取 $m = n$ 且 p 均匀，使每个类型的期望计数为整数；经典设定为 $r = n$ （每个离线顶点一个独立类型），而用于直方图建议实验的**少类型**设定取 $r \ll n$ 。

性能度量为**竞争比**，逐实例计算后再取平均： $\rho(\text{ALG}) = \mathbb{E}[|\text{ALG}|/|\text{OPT}|]$ ，其中 OPT 是**实现实例**的最大匹配，由 Hopcroft-Karp [19] 精确计算。在配对试验 (3.5 节) 下，对逐实例比值取平均是自然的约定：在给定实例上，每个算法都除以同一个精确算出的最优解。它在原则上不同于一项实验研究也可能报告的期望之比 $\mathbb{E}[|\text{ALG}|]/\mathbb{E}[|\text{OPT}|]$ ，因此我们测量了两者的差距：跨越两族预测的九个“算法 $\times$ 质量”格子，两种约定的一致性在 6×10^{-5} 以内（比我们报告的置信区间小两个数量级），故本文没有任何数字依赖于这一选择（`scripts/run_metric_check.py`, A.2 节）。全文的免预测基线是 KVV Ranking，其比值记为 ρ_{base} （**基线强度**）。

以下记号贯穿全文，集中列出以便查阅：

记号	含义	典型取值
n	离线资源数	1000–2000
r	在线请求类型数	$r = n$ （经典）、 $r = 8$ （少类型）
m	一个实例中的到达数	$m = n$
p	类型上的已知分布	除非另述，取均匀
μ	预测的资源度数（度数预测）	—
$\hat{c}$	预测的类型计数（直方图预测）	—

记号	含义	典型取值
k	检验可查看的前缀长度	$k = 200$
η	注入的预测误差强度	$\eta \in [0, 1]$
ρ_{base}	Ranking 在该实例族上的比值 (floor)	0.89–0.99

3.2 算法

所有贪心型算法都是一个原语的实例：给定资源上的一个全序（rank），**GreedyWithPermutation** 将每个到达匹配到其 rank 最小的可用邻居。这使整个算法族成为一条由 rank 参数化的代码路径，这对公平比较与理论都很重要。

- **免预测基线**。SimpleGreedy（恒等 rank）；**Ranking**（均匀随机 rank；基线）；以及随机匹配算法 Feldman 与 Jaillet–Lu，它们通过最大流预计算一个建议匹配：Feldman 用容量 $\{2, 1, 2\}$ 的网络，其单位流子图分裂为蓝、红两个匹配；Jaillet–Lu 用容量 $\{3, 2, 3\}$ 的网络，其分数解给出逐类型的资源尝试顺序表。后两者各有一个**非贪婪**变体（仅跟随建议）与一个**贪婪**变体（回退到任意可用邻居）。
- **度数预测**。MinPredictedDegree（MPD）是按预测度数 $\mu \in \mathbb{R}^R$ 升序排名的 GreedyWithPermutation。常数 μ 给出 Ranking；真实的实现度数给出一致性天花板（MinDegree）。**增强版 Feldman**(MPD) 与 JailletLu(MPD) 仅将 MPD 的 rank 用作贪婪回退的平局打破。
- **类型直方图建议与测试-回退**。由类型上的计数向量 $\hat{c}$ 我们构造一个建议匹配 $\hat{M}$ （ $\hat{c}$ 份拷贝的最大匹配）并记录其逐类型伙伴。**FollowPrediction** 把每个到达送往其类型的预测伙伴；**TestAnd-Match**（Choo 与 BEM）在长度 k 的前缀上做同样的事，检验前缀频率与 $q = \hat{c}/\|\hat{c}\|_1$ 的经验 ℓ_1 距离是否不超过阈值 τ ，据此继续或回退到 Ranking。Chłędowski 式动态**组合器**被移植进来作为基准评测对象，不是贡献。

最后这一点正是第 4 章结构式鲁棒性的全部机制，值得直白说明：因为预测只在若干等价选择之间做决定，承担主要负载的始终是最坏情况最优的基匹配，再差的预测也动不了它。这既是增强算法从不崩溃的原因，也同样是它们在预测很好时吃不到上升空间的原因。

3.3 预测对象与结构化误差模型

各族消费不同的预测对象（度数向量 μ 与类型直方图 $\hat{c}$ ），因此各以自己的旋钮扰动并在并列的面板中报告。对 μ 我们使用四种结构化误差模型，各由强度 $\eta \in [0, 1]$ 驱动，各返回一个 ℓ_1 幅度误差与一个归一化的 **Kendall- τ** 顺序误差：

模型	强度为 η 时的构造	对诱导顺序的影响
随机翻转	以概率 η 独立地把 μ 的一个分量替换为 $[\min \mu, \max \mu]$ 上的均匀抽样	随 η 增大; $\eta = 1$ 即常数质量的预测器, 等价于 Ranking
系统性偏置	单调重缩放 $\mu \leftarrow \mu \cdot (1 + \eta)$	无: 构造上 $\text{Kendall-}\tau \equiv 0$, 而 ℓ_1 幅度线性增长
对抗	反射 η 比例的分量, $\mu(R) \leftarrow (\min \mu + \max \mu) - \mu(R)$	给定比例下最伤顺序的扰动; $\eta = 1$ 使顺序完全反转
分布漂移	朝一个独立抽取的同族图的度数混合, $\mu \leftarrow (1 - \eta)\mu + \eta\mu_{\text{alt}}$	随 η 增大, 但方式是相关的而非逐分量独立的

对 $\hat{c}$ 我们以比例 $\eta \in [0, 1]$ 将实现计数朝一个集中随机目标混合, 报告诱导的 $\ell_1(p, q)$ 。由于 MPD 仅经诱导顺序依赖于 μ , 顺序/幅度之别正是第 5 章的主题。

3.4 一致性与鲁棒性

对一个消费预测的算法, **一致性**是其在完美建议下的比值, **鲁棒性**是其在对抗建议下的最坏比值。由于无法穷举每一种对抗预测, 实验中的鲁棒性以我们所设各扰动档的最小值报告, 它是真实最坏情况的一个上界, 因而是对算法安全性偏宽松的读法。一个有用的算法应既一致、又从不(大幅)低于 ρ_{base} ; 二者之间的张力是第 6 章与 10.2 节展望的主题。

3.5 实验框架

框架是一个小型、依赖轻量的 Python 代码库 (NumPy、SciPy、NetworkX)。每一个随机性来源都取自各自独立、可复现的流, 因此在一次比较中**唯一**变化的是预测, 新增一个算法也从不扰动既有结果(流的清单见附录 A.1)。我们使用**配对试验**: 在一次比较中, 每个算法与误差水平复用同一图、到达序列、OPT 与平局种子, 因此差异可归因于预测本身。所报告的比值为试验上的均值及 95% 正态近似置信区间, 逐算法、逐误差水平计算。

按算法各自报告的区间是保守的取法: 它没有用足配对, 因为两个算法之间的比较其实是一个**配对差**, 只要二者在同一批实例上同向响应, 配对差的区间就更窄。本文做出的每一项比较在这个较宽的口径下都仍然成立(余量最小的一处是 Feldman(MPD) 的 $+0.0044$ 对 ± 0.0023), 因此结论不依赖于采用哪一种口径; 实测的相关系数与配对区间见 A.2 节。

全文每张图与表都由单个脚本从固定种子重生成(附录 A)。

3.6 地基验证：复现 Borodin 等人

在框架上构建基于预测的算法之前，我们对照 Borodin、Karavasilis 与 Pankratov [5] 已发表的实验研究验证了它。这是一次**保真度检验**，而非**贡献**：其目的是确认生成器、i.i.d. 采样器、Hopcroft–Karp 最优解与算法实现能复现该文的定性发现，从而使日后的差异可归因于算法而非基础设施。我们的技术栈 (Python/NetworkX) 不同于该文 (C++/Edmonds–Karp)，故我们以**定性一致**为目标，接受小的绝对差异 (≤ 0.02)。

我们在两个随机图族上复现了核心四算法（六个贪婪/非贪婪变体）， $n = 1000$ 、 $m = n$ 、每参数值 100 次试验（与该文设定一致），单一主种子。

Erdős–Rényi (边概率 c/n ；该文图 9，部分)。我们核对的该文五条定性论断全部复现，绝对差异均低于 0.02。具体而言：扫描 c 复现了特征性的 U 形及其困难情形，SimpleGreedy 在 $c \approx 4.9$ 取最小值 0.864，复杂算法的贪婪变体在 $c \approx 5.3$ 取其最小值 ≈ 0.884 。Ranking 与 SimpleGreedy 无法区分（全部 75 个值上最大差为 0.0017，该文正因此省略 Ranking 的曲线），非贪婪变体随 c 增大单调退化 (Feldman 0.986 $\rightarrow$ 0.730, Jaillet–Lu 0.985 $\rightarrow$ 0.760)。所核对的五条定性论断全部成立（图 3.1）。

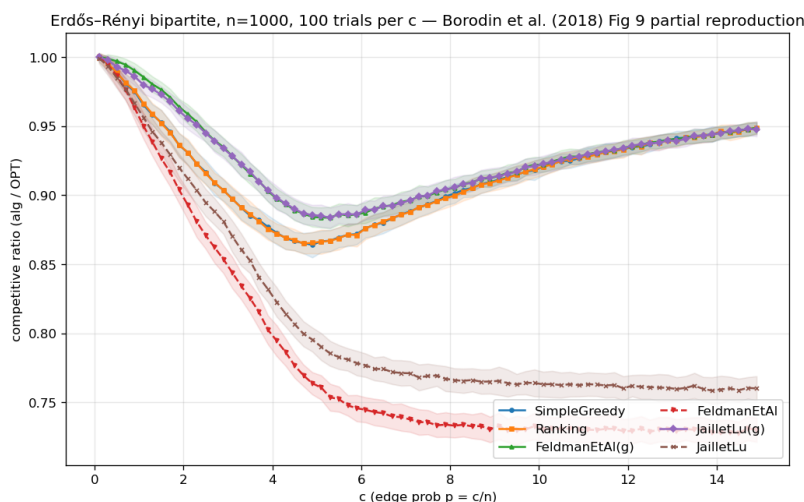


图 3.1: Erdős–Rényi 复现：特征 U 形，贪心最小值在 $c \approx 4.9$ 附近，非贪婪随 c 增大退化。

随机左正则 (左度数 d ；该文图 18，部分；图 3.2)。图景在困难情形 $d = 5$ (SimpleGreedy 最小值 0.890) 重复出现，Ranking $\approx$ SimpleGreedy (最大差 0.0013)，非贪婪随 d 增大退化，贪婪复杂变体渐近 $\approx$ SimpleGreedy。

一个跨族观察。合并两次扫描浮现出一个不平凡的事实：非贪婪的 Feldman 与 Jaillet–Lu 在两个族中收敛到同一渐近比值（分别为 0.730 与 0.760，跨族相差在 0.001 内），且都高于其最坏情况理论界 (0.670 与 0.729) $+0.06$ 与 $+0.03$ 。两个族在这里恰好收敛到同一个值；我们并未考察这是否普适。无论如何，它是本文反复出现的主题（平均情况远比最坏情况温和）的一个早期具体实例，后续各章的预测误差扫描将认真探究它。

结论。框架在两个族上复现了已发表的定性发现，包括困难情形的位置以及 Greedy 与 Ranking 的近

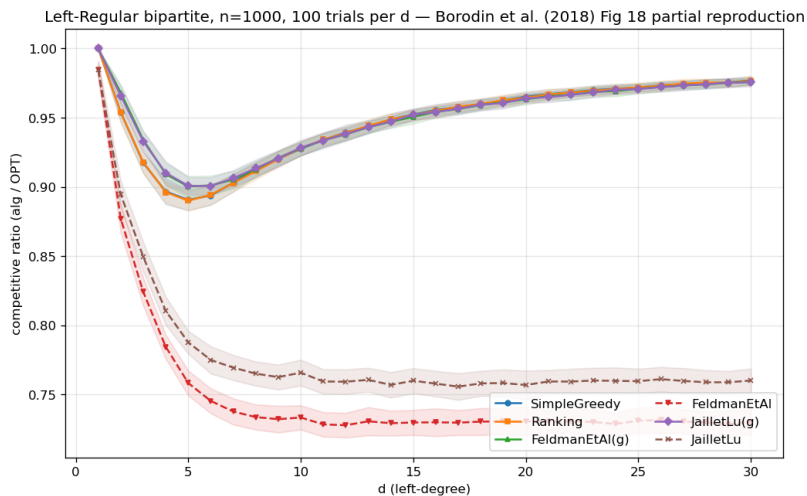


图 3.2: 随机左正则复现: 难点在 $d = 5$, Greedy $\approx$ Ranking。

乎恒等。地基可信; 第 4-9 章的贡献建立于其上。(完整的复现表与论断核对清单见附录 A。)

第四章 统一基准

在验证框架（第 3 章）之后，我们将三个算法族置于其上，确立本文的组织性发现。本章以竞争比（均值 $\pm$ 95% 置信区间）作为预测质量的函数，在三个面板中报告，各面板对应各族被设计的体制；它所提炼的四个发现（4.2 节）贯穿全文其余部分。

4.1 设计

各族消费不同的预测对象，故各以自己的扰动旋钮驱动并在并列面板中呈现（第 3 章）；共享的框架（图、OPT、置信区间方法、以及免预测地板）正是使各面板可比的东西。

实例格式与符号。每个面板都使用第 3 章的实例格式： n 个离线资源、 r 个在线请求类型、以及从类型分布独立同分布抽取的 m 个到达；本章全程取 $m = n$ ，即请求数与资源数持平。各面板仅在连接请求与资源的类型图上不同；正是这一处变化，把输入在两个预测族各自被设计的体制之间切换。三个面板的选取使**度数**预测器分别拥有强信号、弱信号和完全无用武之地：

- **面板 A：重尾度数 (Zipf)** ($n = 1000$, 60 次试验)：资源度数服从指数为 1.0 的 Zipf 幂律，因此少数资源被激烈争用、多数资源少有请求可用。这种重尾分布让**度数**预测器有真实的信号可携带。
- **面板 B：左正则 $d=5$** ($n = 1000$, 60 次试验)：每个到达的请求恰好连接 $d = 5$ 个均匀随机的资源，资源度数近乎均匀，即第 3 章的困难情形，度数预测器几乎没有信号可携带。
- **面板 C：少类型 $r=8$** ($n = 2000$, 50 次试验)：只有 $r = 8$ 个不同的请求类型，每个类型平均到达 $\approx n/r = 250$ 次，即近乎完美可匹配的少类型体制，正是**直方图**建议算法被校准的场景；其测试-回退检验考察前 $k = 200$ 个到达。

因此面板 A 与 B 检验**度数**预测族（MPD 及其增强），面板 C 检验**直方图**建议族（FollowPrediction、TestAndMatch、组合器）。

共享方法。配对试验、独立随机流与置信区间的处理与 3.5 节完全一致；面板专属参数即上文所列，置信区间全程很紧 (± 0.001 - 0.003)。质量列实例化 3.3 节的误差模型。**度数**面板：**完美**（真实实现度数）、**含噪**（强度 $\frac{1}{2}$ 的随机翻转）、**对抗**（逆序反射）、**垃圾**（独立随机 μ , $\equiv$ Ranking）；**建议**面板：**真实**直方图以 $\eta \in \{0, 0.3, 0.6, 1.0\}$ 朝一个集中随机目标混合（**完美/轻度/严重/垃圾**）。两套列**不可通约**（它们扰动的是不同的预测对象、用的是不同的旋钮），因此只有面板内部的比较才有意义。**表 4.1** 将各面板的比值与其柱状图并排呈现；发现见 4.2 节。

面板 A: 重尾度数 (<i>Zipf</i>)	完美	含噪	对抗	垃圾
Ranking (地板)	0.948	—	—	—
MinDegree (预言机)	0.996	—	—	—
MPD	0.989	0.956	0.908	0.946
Feldman(MPD)	0.981	0.979	0.976	0.978
JailletLu(MPD)	0.977	0.976	0.974	0.975
Feldman (基础版)	0.887	—	—	—
JailletLu (基础版)	0.900	—	—	—

面板 B: 左正则 $d=5$	完美	含噪	对抗	垃圾
Greedy = Ranking (地板)	0.890	—	—	—
MinDegree (预言机)	0.966	—	—	—
MPD	0.932	0.906	0.854	0.888
Feldman(MPD)	0.906	0.902	0.896	0.900
JailletLu(MPD)	0.904	0.903	0.899	0.901
Feldman (基础版)	0.760	—	—	—
JailletLu (基础版)	0.788	—	—	—

面板 C: 少类型 $r=8$	完美	轻度	严重	垃圾
Ranking (地板)	0.990	—	—	—
MinDegree (预言机)	0.999	—	—	—
FollowPrediction	1.000	0.832	0.679	0.472
TestAndMatch (Choo)	1.000	0.984	0.989	0.990
TestAndMatch (BEM)	0.998	0.988	0.988	0.968
Combiner (基准)	0.990	0.990	0.990	0.990

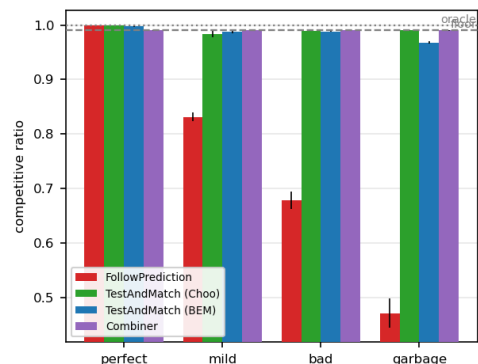
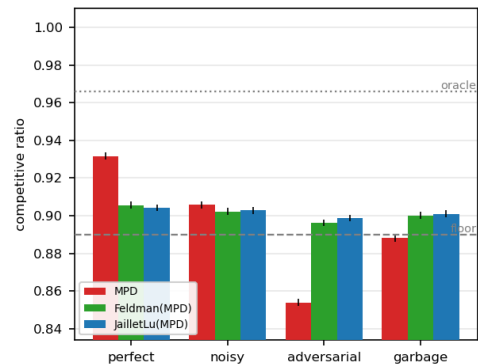
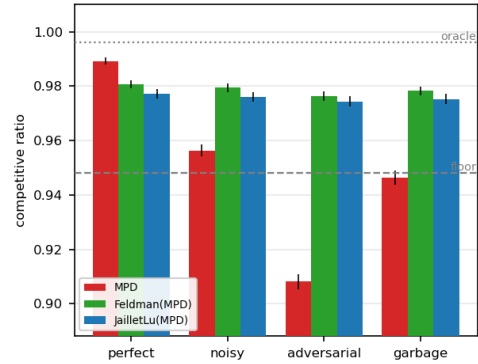


表 4.1: 统一基准。各面板的竞争比（配对试验均值；所有 95% 置信区间 ≤ 0.003 ）与其柱状图并排（误差棒：95% CI；虚线：免预测地板；点线：预言机天花板）。加粗标出每个无防护预测跟随者最差的一列，三者都跌破了各自面板的地板。该地板是实例相关的（它是 Ranking 在该面板自己图族上的比值，不是一个常数），因此三个面板的地板（0.948、0.890、0.990）彼此不可比，两族预测的质量列同样不可比（3.3 节）。

4.2 四个发现

(F1) 鲁棒性是工程出来的，而非免费的：裸跟随者跌破地板。两个无防护的预测跟随者一旦预测为对抗或垃圾，都会跌到免预测 Ranking 地板之下：MPD 落到 $0.908 < 0.948$ （面板 A）与 $0.854 < 0.890$ （面板 B），FollowPrediction 崩到 $0.472 \ll 0.990$ （面板 C）。也就是说，在对抗或垃圾建议下，无防护地使用二者中的任何一个都劣于不用预测。表中每个鲁棒算法（增强版、TestAndMatch、组合器）都

在构造上避免了这一点。

(F2) 两种不同的鲁棒机制，形状不同。 结构式鲁棒 (Feldman(MPD)、JailletLu(MPD))：最坏情况最优的基匹配承担负载，预测仅打破平局，故性能几乎平坦：Feldman(MPD) 从完美到对抗仅移动 $0.981 \rightarrow 0.976$ (面板 A)。它不会崩，但也封顶了上升空间 (永远够不到 0.996 的预言机)。自适应鲁棒 (TestAndMatch)：在亚线性前缀上检验，再提交，建议好时抓住上升空间 (Choo 1.000)，建议坏时守住地板 (0.990)。在面板 C 上，它是唯一在两端都处于上包络的算法。两种机制以相反的方式用一致性换鲁棒性；图 4.1 把每个算法画在一致性-鲁棒性平面上，两种机制相反的取舍，以及 TestAndMatch 之外通向理想右上角的空白区域，一目了然。

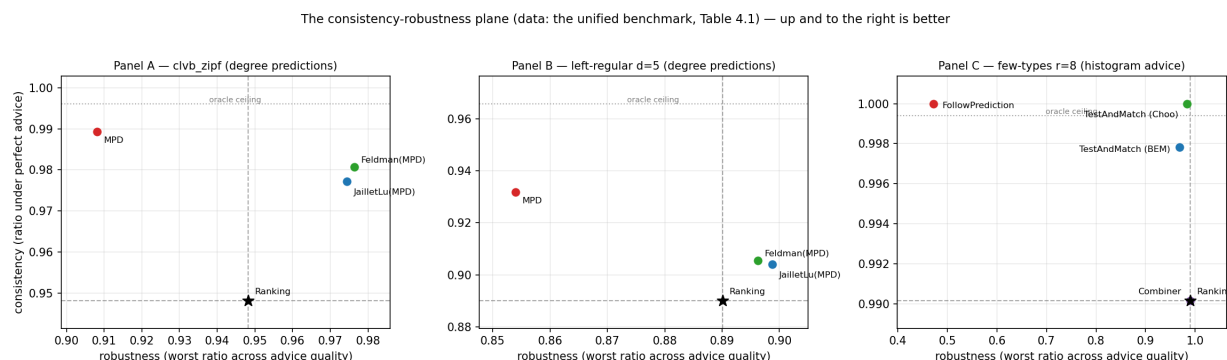


图 4.1: 一致性-鲁棒性平面 (数据即表 4.1)。横轴：完美建议下的比值；纵轴：最差扰动档下的比值，即 3.4 节的操作化鲁棒性。虚线为免预测地板，点线为预言机天花板；TestAndMatch 最接近理想的右上角。

(F3) 平均情况输入上一致性上升空间很小；差异全在坏建议一侧。 在少类型上，免预测 Ranking 已达 0.990，用真实度数的 MPD 达 0.999：整个一致性余量是 0.009 ± 0.001 ，比本文后面测到的多数效应都小。面板 C 中每个宽大的裂口都是下行裂口。这是本文论点的一图缩影；第 6 章表明为何没有负担得起的检验能逃脱它，结论章的展望 (10.2 节) 解释它为何是被迫的。

(F4) 增强救活了结构性薄弱的基算法。 Feldman 与 Jaillet-Lu 为最坏情况比值而调，在这些平均情况输入上是最弱的免预测项 (面板 B: $0.760 / 0.788$ ，低于 Greedy 的 0.890)；MPD 增强将它们抬到 ≈ 0.90 。预测对为最坏情况设计的算法所做的，多于对贪婪所做的，统一的表格让这一配对一目了然。

4.3 本章小结

预测在这里的价值是下行保护，而非性能。接下来要问的是：剩下的那点损失由什么支配。第 5 章表明，支配它的不是预测误差的大小，而是它诱导的顺序。

第五章 什么支配损失：顺序误差，以及已知的界没有说的事

第 4 章表明，在平均情况输入上，度数预测算法的损失很小。本章追问是什么支配该损失。MinPredictedDegree 按预测度数升序匹配，故它仅通过 μ 在资源上诱导的顺序依赖于预测器：诱导相同顺序的两个预测器产生相同的匹配，而 μ 的单调重缩放毫无影响。因此正确的问题不是“误差有多大”，而是“哪个顺序误差支配损失、以及有多紧”。回答它需谨慎，因为“支配 MPD 的是顺序而非幅度”这一确定性事实已是定理，我们对何为已有、何为我们的贡献保持明确。

5.1 已知的部分 (ACI)

Aamand、Chen 与 Indyk [11, 附录 D] 证明：在 CLV-B 模型上，MinPredictedDegree 相对真实期望度数的匹配损失至多为 $n - \text{LIS}(w[\mu])$ 。这里 w 是真实期望度数向量（记作 w 而非 p ，是为了与 3.1 节的类型分布 p 区分开）， $w[\mu]$ 是该向量按 μ 诱导的顺序排成的序列，LIS 是它最长非降子序列的长度：一个纯顺序量。特别地，一个单调（保序）预测器使 $w[\mu]$ 已排好序，故 $n - \text{LIS} = 0$ ，损失为零。因此，顺序依赖性与单调偏置的零效应是 ACI 的结果，而非我们的；我们的系统性偏置误差模型（单调重缩放）在构造上 $\text{Kendall-}\tau \equiv 0$ ，且相应地在整个基准中使 MPD 的比值完全不变（第 4 章）：这是对 ACI 陈述的经验印证，而非新发现。

5.2 我们的刻画

我们跨强度扫描四种结构化误差模型，并在同一批实例上逐模型、逐水平记录三个量：实际 MPD 匹配损失、ACI 的 $n - \text{LIS}$ 、以及归一化 Kendall- τ 顺序误差，它报告在 $[0, 1]$ 上，0 表示预测的顺序完全正确，1 表示完全反序（图 5.1； $n = 1000$ ，Zipf 指数 1.0，40 次试验）。

(i) **ACI 的 $n - \text{LIS}$ 界正确但非常松。**实际损失在每一点都远低于该界。以各模型最强扰动档上“界除以实际损失”计，它松了约 $16\times$ （对抗）到 $75\times$ （分布漂移）；图 5.1(a) 中所有点都紧贴坐标轴。

(ii) **$n - \text{LIS}$ 饱和，无法区分误差结构。**对每种非平凡误差，它都被钉在其最大值附近（ $\approx 610\text{--}673$ ，而可能的最大值是 $n = 1000$ ，对照的实际损失只有 8 到 42），尽管各模型的真实损失相差五倍（漂移 8.1、

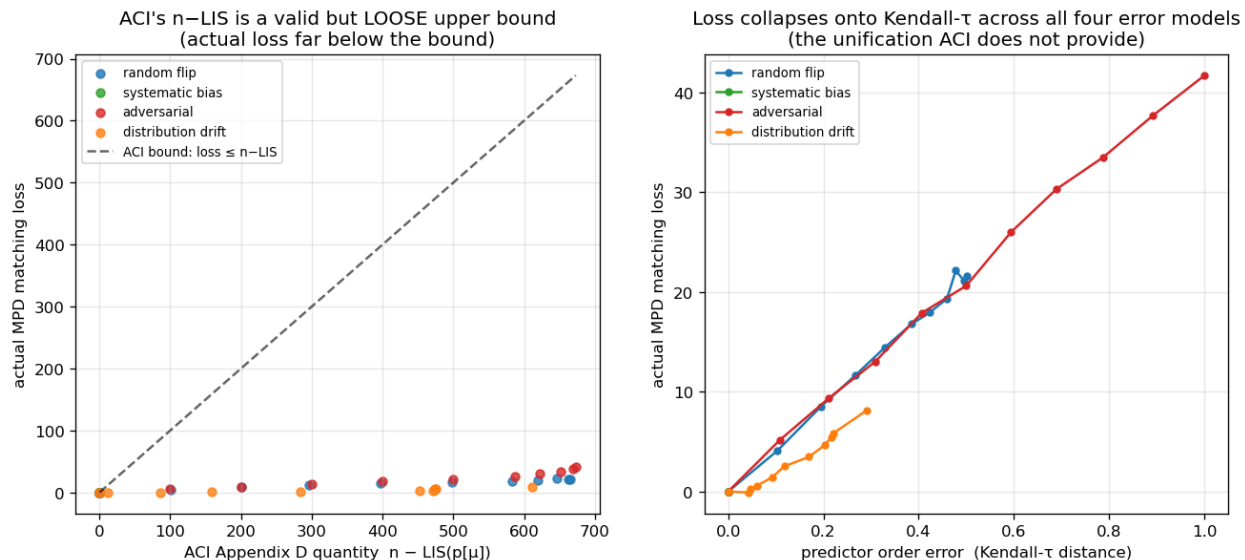


图 5.1: 顺序误差支配损失: 实际损失远低于 ACI 的 n -LIS 界 (a), 且四种误差模型都塌缩到 Kendall- τ 上 (b)。

随机翻转 21.6、对抗 41.7)。作为一个几乎总是 $\approx n$ 的界，它几乎不携带“哪个预测更有害”的信息。

(iii) **Kendall- τ 是支配性的顺序度量，各模型塌缩到它之上。**对 Kendall- τ 作图（图 5.1(b)），四个模型落在一条递增曲线上：损失随 τ 上升（漂移、随机翻转、对抗为 $0.29 \rightarrow 0.50 \rightarrow 1.0$ ，对应损失 $8.1 \rightarrow 21.6 \rightarrow 41.7$ ），系统性偏置钉在 $\tau = 0$ 、零损失。这一塌缩不只是目测：把四个模型、所有误差水平合并（44 个点），Kendall- τ 与实际损失之间的秩相关为 Spearman $\rho = 0.979$ (Pearson $r = 0.992$)，在每个非退化模型内部也都在 0.97 以上。**预测**损失并统一各误差结构的量，是 Kendall- τ 顺序距离，而饱和的 $n - \text{LIS}$ 无法分辨它。

5.3 本章小结

本章所添加的（顺序依赖性本身是 ACI 的结果，见 5.1 节）是对“已知的界究竟说了多少”的精确经验刻画：ACI 的 $n - \text{LIS}$ 界松了一到近两个数量级且会饱和，而 Kendall- τ 预测实际损失并将四种误差结构统一起来。这既锐化了理论的实践内涵，也为在真实数据上对预测器施压时所用的单一顺序误差轴提供了依据。它同时解释了为什么第 7 章可以仅凭顺序保真度来说明一个廉价历史预测器为何有效，以及第 8 章为何会尝试直接为顺序训练一个预测器，并最终失败。

第六章 深入研究测试-回退

测试-回退算法是第 4 章的自适应鲁棒机制。本章分四部分给出对它们的首个实证研究：它们所达到的鲁棒性包络（6.1 节）、接受阈值的一个反直觉失效（6.2 节）、其重校准（6.3 节），以及对动态组合器的基准评测（6.4 节）；后者解释了先测试再一次性提交的结构。6.3 节中重校准所暴露的分辨率极限，是本章通往结论章的接口：它在整个难度范围上持续成立，而 10.2 节对它做定量解读。

全章有两套命名约定。**FollowPrediction** 是无防护的跟随者，它不做检验就模仿建议匹配；**TestAndMatch** 泛指测试-回退方案，**Choo** 与 **BEM** 是它的两个已发表实例。二者共享“先测试再提交”的结构，区别在于接受阈值如何设定。 ℓ_1 检验是原作者自己也退回使用的经验代理（第 3 章）。

6.1 鲁棒性包络

在少类型实例上扫描建议误差 $\ell_1(p, q)$ ($n = 2000$ 个到达, $r = 8$ 个请求类型, 受检前缀为 $k = 200$ 个到达, 40 次试验; 图 6.1), **FollowPrediction** 从完美建议的 1.000 线性退化到 $\ell_1 \approx 1.1$ 处的 0.453, 远低于免预测地板 (≈ 0.99)。TestAndMatch 则稳在上包络: 建议好时抓住收益, 建议坏时其前缀检验拒绝它并回退到 Ranking, 在这次扫描的任何一点上都不跌破免预测地板 (BEM $0.998 \rightarrow 0.969$; Choo $1.000 \rightarrow 0.991$)。这是第 4 章结构式鲁棒的自适应对应物。

6.2 一个在平均情况输入上过于宽松的阈值

在边界建议 ($\eta = 0.15$, 真实 $\ell_1 \approx 0.16$, 检验最吃力之处, 见图 6.2) 处扫描前缀 (检验) 规模 k , 一个更大、更准确的检验反而做出更差的决策: 随 k 从 25 增到 800, 比值下降 $0.992 \rightarrow 0.956$, 误判率上升 $0.00 \rightarrow 0.60$ 。

机制分两半; 阈值是 Choo 与 BEM 自己的, 下面给出的是我们对它在设计点之外如何表现的刻画。第一, 接受阈值 τ 是按 β 校准的, 即免预测基线被证明能达到的竞争比, 这里 $\beta \approx 0.696$, 是随机到达序下 Ranking 的最坏情况比值, 也是 Choo 等人给阈值取的值 [12]。而在这些实例上, 实际基线是 ≈ 0.99 (F3), 故经验盈亏平衡点位于 $\ell_1 \approx 0$, 远低于 τ 。第二, 前缀在扫描的两端以相反方向与这一落差发生作用。一个小而含噪的前缀高估 ℓ_1 、拒绝了边界建议, 安全落在地板上, 但它对得莫名其妙: 拒绝是因为自己噪声大, 而不是因为建议坏。一个大而准确的前缀正确测得 $\ell_1 \approx 0.16 < \tau$, 于是照章接受了最

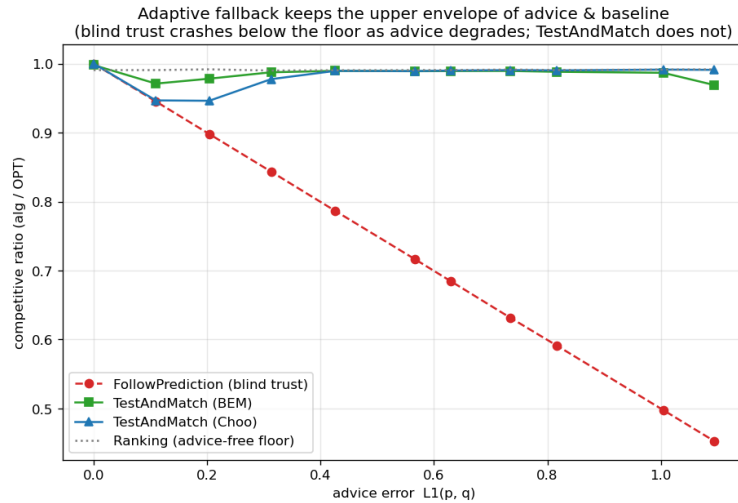


图 6.1: 鲁棒性包络: 随建议误差增大, 盲目 FollowPrediction 跌破免预测地板; TestAndMatch 保持在上包络。

坏情况阈值判定为可接受的轻度坏建议, 表现低于基线。换句话说, 在强基线输入上, 阈值是按错误的基线校准的, 而前缀规模只决定算法执行它有多忠实。

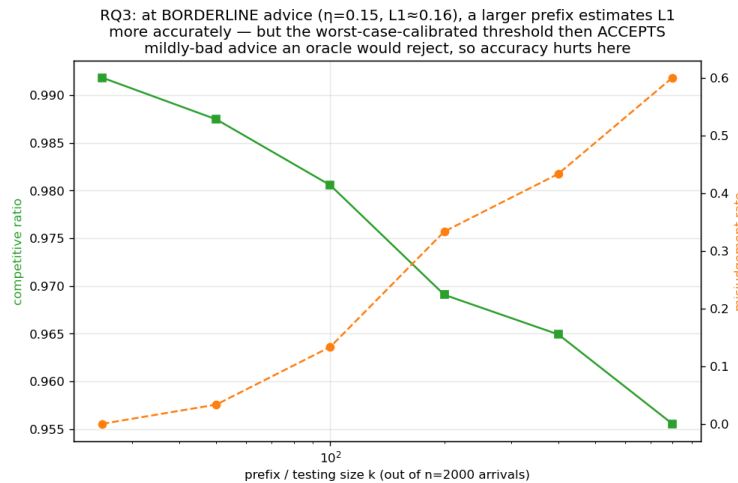


图 6.2: 边界建议下的检验代价: 在最坏情况校准的阈值下, 更大、更准的前缀检验反而做出更差的决策。

6.3 重校准, 及其暴露的分辨率极限

所谓检验的分辨率, 指的是在前缀长度 k 下, 其经验估计器能把多小的 ℓ_1 差别与自身的采样噪声区分开; 本节讲的就是这个量及它带来的极限。将 τ 重校准到测得的基线 $\hat{\beta}$ 可消除该病理 (图 6.3): 在边界建议处, 最坏情况阈值的误判随前缀增长从 0.03 攀到 1.00、比值降到 0.920, 而重校准阈值在每个

前缀规模上都保持误判 0.00、比值 ≈ 0.986 。但重校准暴露出一个更深的极限。在完美建议下，最坏情况阈值得 1.000，而重校准的仅得 0.987：重校准的 $\tau \approx 2(1 - \hat{\beta}) \approx 0.028$ 小于经验 ℓ_1 估计器自身的噪声底：在完美建议下，长度 k 的前缀的类型频率与真实直方图之间的 ℓ_1 距离，也就是纯采样噪声；它随 k 增大而下降，在本节扫过的前缀长度上从 ≈ 0.13 降到 ≈ 0.05 ，故它永远无法自信地接受：它拒绝一切（包括完美建议），总是回退到基线。一言以蔽之：

在强基线实例上，在这种形式的阈值里、在我们负担得起的前缀长度上，没有一个能既抓住一致性上升空间又保持安全：上升空间很小、且位于估计器的分辨率之下。最坏情况阈值过度接受；重校准阈值过度拒绝；更好的检验只是更忠实地跟随其中之一。

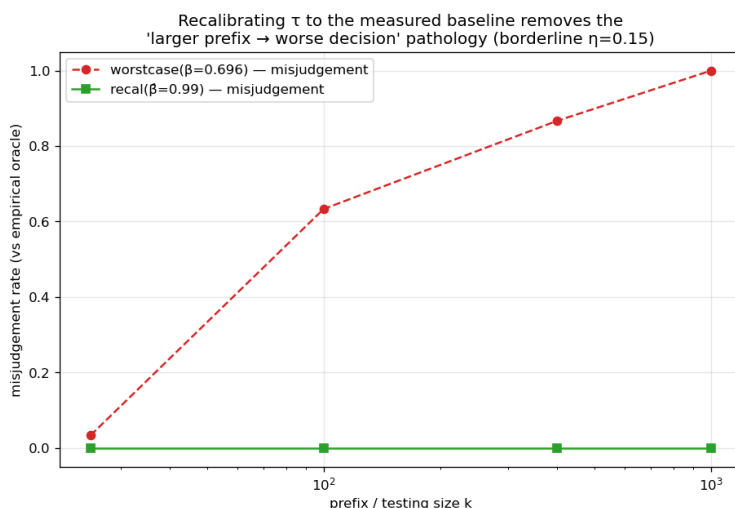


图 6.3: 重校准消除阈值病理：误判率在每个前缀规模都保持为零，而最坏情况阈值下攀向 1.0。

图 6.4 把这一发现从单个阈值推广到整个难度范围。扫描类型数（设定基线强度的旋钮）可见，随着基线变弱，完美建议的潜在上升空间增大；但一个亚线性前缀检验能安全捕获的上升空间始终钉在零附近：凡上升空间存在之处，经验检验的分辨率（噪声底）都远高于盈亏平衡边际。因此本章的墙并非一个校准不当的阈值。上升空间与检验分辨率同进退，而结论章的展望（10.2 节）将给这一耦合一个定量形式：做出决策所需的前缀规模按赌注的平方反比增长。

6.4 动态组合器被支配，并揭示为何匹配需要先测试再提交

最后我们对 Chłędowski 式动态组合器做基准评测，以映衬测试-回退的一次性提交结构。在其稳健调参下，组合器恰好落在 6.1 节少类型族的地板上（所有建议质量下均为 0.990）：它从不崩溃，但一点一致性上升空间也抓不到，严格被 TestAndMatch 支配。更有启发的是，一个急切、中途切换专家的组合器揭示了一个不可逆问题特有的惩罚：在一个规模更小、用作机制检查的实例上（ $n = 600$ ， $r = 6$ ，单个种子、不做平均，是附录 A.3 的手工可验证脚本之一），完美建议下急切切换得 0.927，低于纯跟随者（1.000）和同一实例上的 Ranking（0.958），因为中途从 Ranking 切到跟随建议会使已提交的匹配落入一个不兼容的混合态。由于只有一个实例，这个数字用于说明机制而非测量其大小；另需注意其中

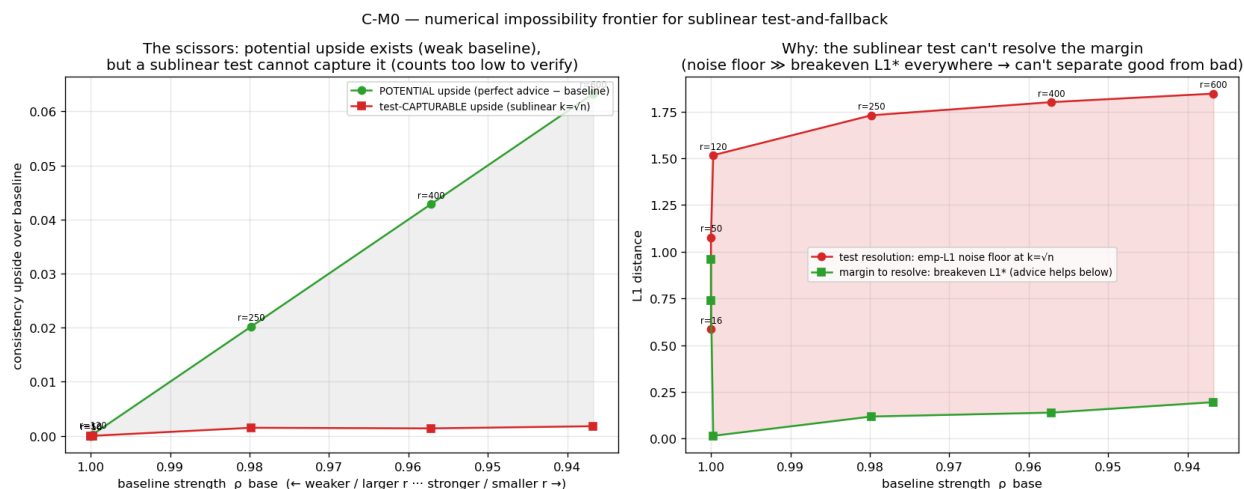


图 6.4: 检验之墙的边界图。横轴：基线强度 ρ_{base} ，由请求类型数扫出（越靠左基线越弱、 r 越大）；纵轴：相对基线的一致性上升空间。随基线变弱，完美建议的潜在上升空间增大，但亚线性检验能安全捕获的上升空间始终接近零：凡上升空间存在之处，经验检验的分辨率都远高于盈亏平衡边际。

的 0.958 是 Ranking 在那个实例上的值，不是上面那个族的 0.990 地板。在不可逆问题中，跟随/回退的决策必须在大批承诺之前做出，这正是 Choo/BEM 在前缀上检验、然后提交、而非像缓存式组合器那样动态切换的原因。对缓存而言是廉价保险的动态组合器，无法干净地移植到匹配上。

6.5 本章小结

测试-回退提供了 6.1 节的自适应鲁棒包络，但其接受/拒绝决策在强基线输入上有根本限制：没有经验 ℓ_1 阈值能既抓上升又保安全（6.3 节），且动态切换被“先测试再提交”支配（6.4 节）。6.3 节的分辨率极限，及其在整个难度范围上的持续成立（图 6.4），是 10.2 节理论展望的经验锚点；图 6.4 是本章最该记住的一张图。下一章要问的是：这一切在合成实例之外是否依然成立。

第七章 外部效度

第 4-6 章使用合成图与一个合成误差旋钮。这一图景是真实的吗？我们从三个方面施压，每一处用真实对应物替换掉一种合成成分：7.1 节把合成误差换成真实 trace 上的时间漂移，7.2 节把合成图换成六个真实图，7.3 节把手工构造的预测器换成一个学习得到的预测器。

7.1 一个真实、廉价的预测器

我们用最廉价的现实预测器替换合成旋钮：上一窗口的历史统计。真实的 Wikipedia 每日页面浏览给出一个直播日（真值）与若干更早的日（1、7 或 30 天陈旧的预测），故误差是真实的时间漂移；我们将该 trace 映射到一个固定的服务拓扑，并通过度数路线（MPD）消费该预测（图 7.1）。本章全程用**缺口捕获率**衡量一个预测器实现了多少可得收益： $(\rho_{\text{ALG}} - \rho_{\text{base}})/(\rho_{\text{oracle}} - \rho_{\text{base}})$ ，即它填平了“基线到预言机”这段缺口的多大比例。三个事实浮现。

第一，**这个预测器很廉价**。带预测的文献通常设想一个昂贵的学习模型；这里它只是一次线性时间的计数，每个实例约 0.11 毫秒，而计算一次 OPT 约 4.4 毫秒，占比只有百分之几。这是全文仅有的两个墙钟数字，因而依赖机器；其余一切都是比值。

第二，**收益真实、部分、且从不有害**：一个陈旧预测达到 27%–68% 的缺口捕获率（随陈旧度下降），且**始终高于免预测基线**（0.938–0.957 vs 0.923–0.925，即便在 30 天时）。

第三，原因在于：**拓扑聚合使廉价预测器保序**。诱导度数预测器的顺序误差仅为 $\text{Kendall-}\tau \approx 0.19 - 0.32$ ，约为原始直方图（0.38–0.49）的一半；而由于 MPD 仅依赖顺序（第 5 章），聚合后的度数路线在真实漂移下存活下来。将**同一预测**通过原始直方图消费则是灾难性的：盲从的 FollowPrediction 崩到 $0.68 \rightarrow 0.36$ ，远低于 ≈ 0.92 的基线：这正是第 4 章与第 6 章的鲁棒算法的用武之地。

7.2 六个真实世界图

我们在六个 Network Repository 图（两个 Facebook 社交、两个 *C. elegans* 生物、两个经济投入产出）上，跨相同的质量列、带 95% 置信区间，重跑第 4 章的度数预测算法阵容（图 7.2）。两个承重发现是普适的。**F1 在全部六个图上成立**：喂给对抗预测器的裸 MPD 在每个图上都跌破 Ranking 地板，差

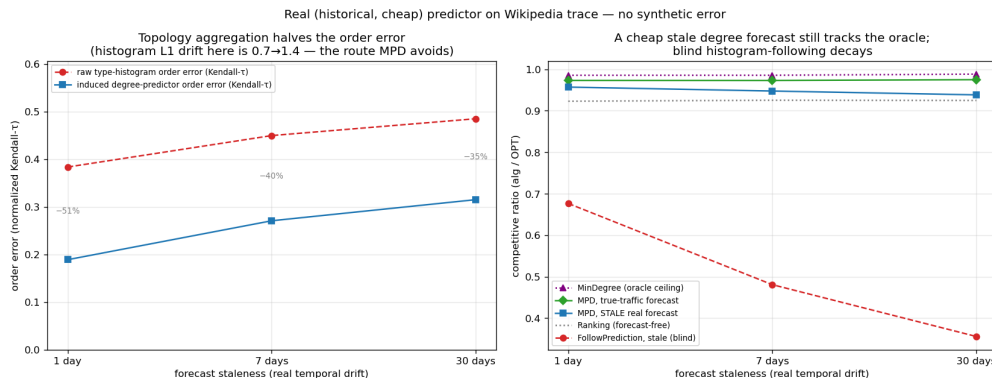


图 7.1: 本章最该记住的一张图，真实的廉价预测器（Wikipedia trace）：聚合使顺序误差减半（a），度数路径经受住陈旧化（b）；盲目跟随直方图则衰减。

0.06 (Reed98) 到 0.10 (CE-PG)。**F3** 在全部六个图上成立，并印证其自身逻辑：一致性上升空间处处很小（均值 +0.049；范围 +0.022~+0.077），且恰恰在基线最强处最小：两个稠密经济图，其 Ranking 已达 0.965/0.977，给出最小的上升空间。

结构式鲁棒发现 **F2** 在全部六个图上定性成立（增强版跨质量列的谱宽在每个图上都小于裸 MPD），并按“谱宽不超过 MPD 一半”这个更强的判据在四个社交/生物图上严格成立（谱宽为 MPD 的 0.22~0.29 $\times$ ）；在两个稠密经济图上保护只是部分的：增强缓冲了对抗下的跌落（0.939 vs 裸 MPD 的 0.893），但清不掉异常高的 0.965 地板，落到其下 ≈ 0.03 。那两个图太稠密，匹配近乎平凡（Ranking ≈ 0.97 ，MinDegree = 1.00），故既无上升空间可抓（F3）、也无多少下行需保护：这个边界正是 F3 自身机制在起作用，而不是它的例外。最后，F4 在此极为显著：为最坏情况设计的 Feldman/Jaillet-Lu 在经济图上是弱的最弱的免预测项（0.73~0.77），而增强将它们抬到 0.99，一次 +0.26 的救援。

7.3 学习预测器是否有帮助？

由于 MPD 仅经顺序消费预测器（第 5 章），人们或许会想用顺序损失（rank loss）而非回归来训练它。我们对此做了检验，报告一个诚实的负结果：这一优势在刻意构造的特征上确实存在，但在真实时序特征上消失。rank 训练与回归训练的预测器诱导出相同的顺序、达到相同的比值。实验、数字以及背后的三步论证见 8.1 节；对外部效度而言，重要的只是它的推论。一旦预测器保序（而廉价的历史计数本已如此，见 7.1 节），在平均情况匹配上，无论更好的算法还是更好训练的预测器都买不到多少。

7.4 本章小结

在真实预测器、真实图与一个学习预测器上，同一堵墙依然矗立：免预测基线近乎最优、无防护跟随不安全、预测买到的是下行保护而非性能，只有一个具启发性的边界，即 7.2 节那两个稠密经济图，增强在那里清不掉异常高的地板。在我们所能触及的每一种设置上都从经验上确立了这堵墙之后，第 8 章

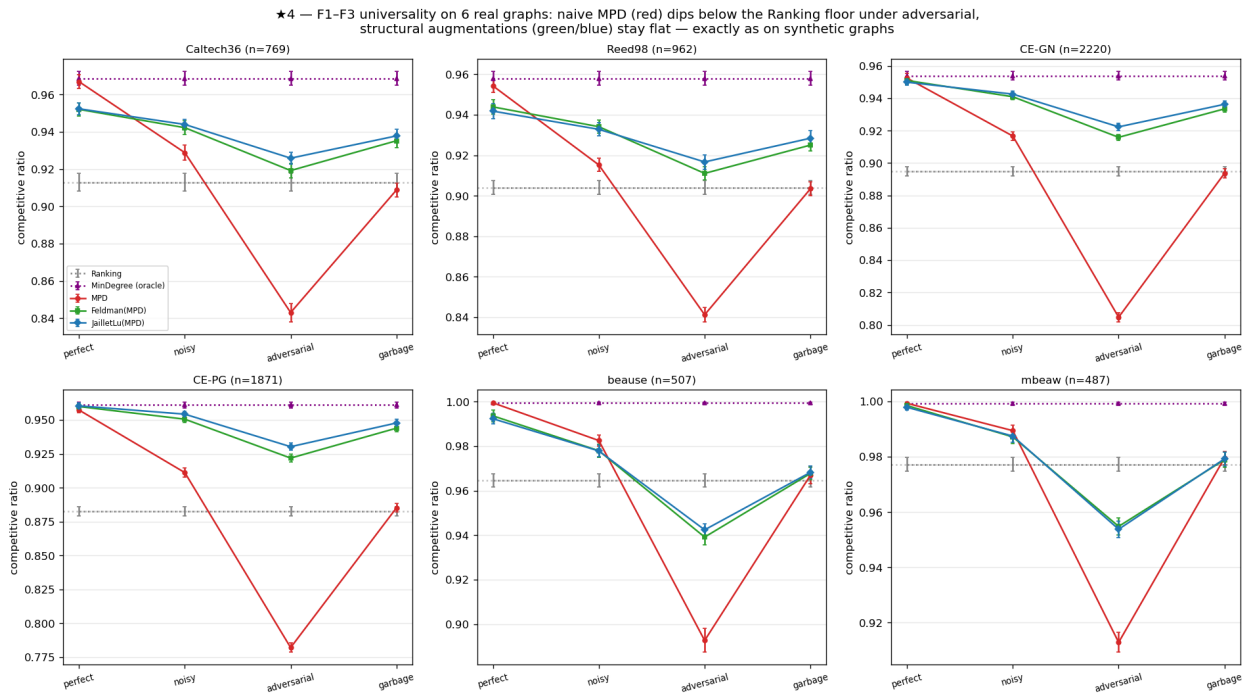


图 7.2: 六个真实图上的 F1-F3, 每图一个子图 (各子图的纵轴范围不同, 它们的地板本就不同, 因此请对照各自的虚线地板来读, 而不要跨子图比较)。裸 MPD (红) 处处跌破 Ranking 地板; 结构式增强 (绿/蓝) 保持平坦。

报告试图越过它的方向，结论章的展望（10.2 节）解释它为何是被迫的。

第八章 探索方向与负结果

实验各章（第 4–7 章）确立了一堵墙：在平均情况匹配上，免预测基线近乎最优，故预测是鲁棒性保险而非性能杠杆。在把这堵墙当作定论之前，我们追求了三个各自**试图越过它**的方向：学习预测器以榨取更多性能（8.1 节）、寻找一个预测真正有帮助的服务场景（8.2 节）、以及让一次系统的文献综述为我们指出最高价值的贡献（8.3 节）。三者都返回了同一个答案，它们的收敛正是使我们怀疑这堵墙是被迫的原因。以下逐一给出，并附上终结该方向的证据。

8.1 学习为预测器排序（一个负结果）

第 5 章表明 MPD 仅通过其诱导的**顺序**消费预测器。这提示了一个具体的改进途径：不再训练预测器最小化其对真实度数的**回归**误差（标准的“先预测再优化”目标），而是用一个**顺序感知**损失（成对 rank 损失），训练它，使其优化算法实际使用的量。我们分三步考察。

M0：机制存在（图 8.1）。用刻意**发散**的合成特征（一个特征承载幅度、另一个承载顺序），一个 rank 训练的线性预测器在决策度量上锐利地击败回归训练者：匹配比值 0.989（基本上是预言机）对 0.974，而 rank 训练的预测器对真值的回归误差**更差**（MSE 87.6 vs 33.4），顺序却更好（Kendall- τ 0.058 vs 0.255）。这一解离是真实的：**更差的拟合给出更好的决策**。因此，只要特征把幅度与顺序分开，回归就是错误的训练目标；而下面的 M1 与 M3 表明，这个条件很少成立。

M1：优势被双重门控，且很小。扫描特征发散度与图难度，rank 优势在特征不诱导顺序/幅度冲突时为零，在基线本已最优的容易实例上也为零（又是那堵墙）。它在合成图上峰值仅 +1.3%；若以缺口捕获率（7.1 节）而非绝对比值来衡量，rank 训练基本上恢复整个预言机缺口，而回归留下约 30% 未实现，但缺口本身很小。

M3：它在真实特征上消失（图 8.2）。决定性的检验使用来自真实服务 trace 的真实时序特征（各资源在此前若干窗口的引用计数）来预测下一窗口（150 个上下文长度类型、500 个度数为 8 的副本、40 个窗口、三个滞后特征、训练/测试按 60/40 划分）。此处 rank 训练与回归训练的预测器产生**相同**的顺序（Kendall- τ 0.126 vs 0.126）与相同的匹配比值。驱动 rank 训练的**顺序/幅度发散**是**人为**特征的属性；现实的滞后计数特征是同一流行度的共线含噪估计，故回归本已把顺序学得和排序一样好。

裁决。用决策对齐损失学习预测器并不改变第 4–7 章的图景，我们把它作为一个负结果报告：其取胜需要一个现实中不出现的特征发散，且即便出现，收益也受制于那（微小的）基线到预言机缺口。该结

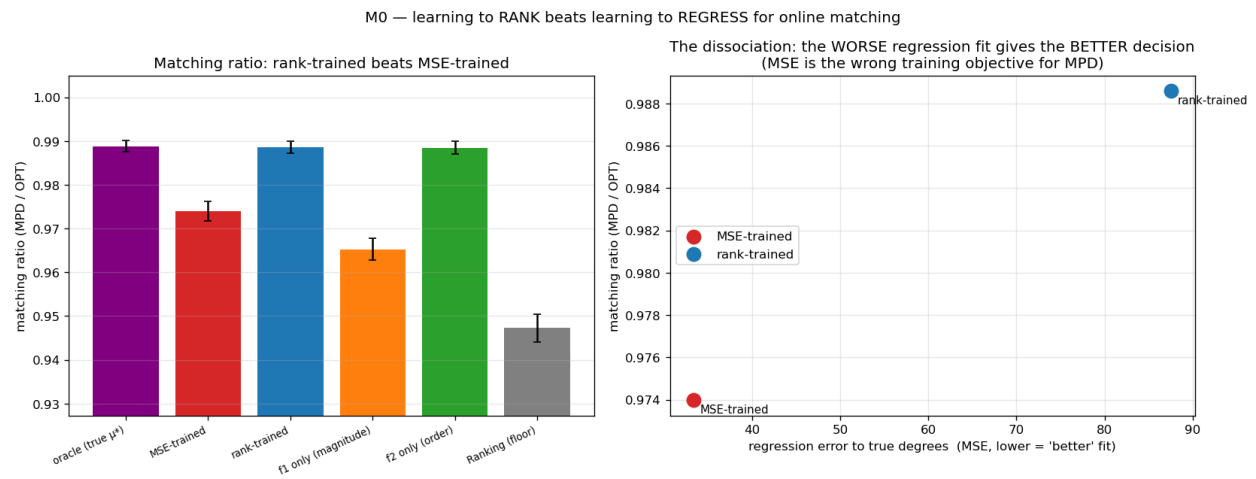


图 8.1: M0: 发散特征下, rank 训练在匹配比值上胜过回归 (左), 尽管回归拟合更差 (右)。

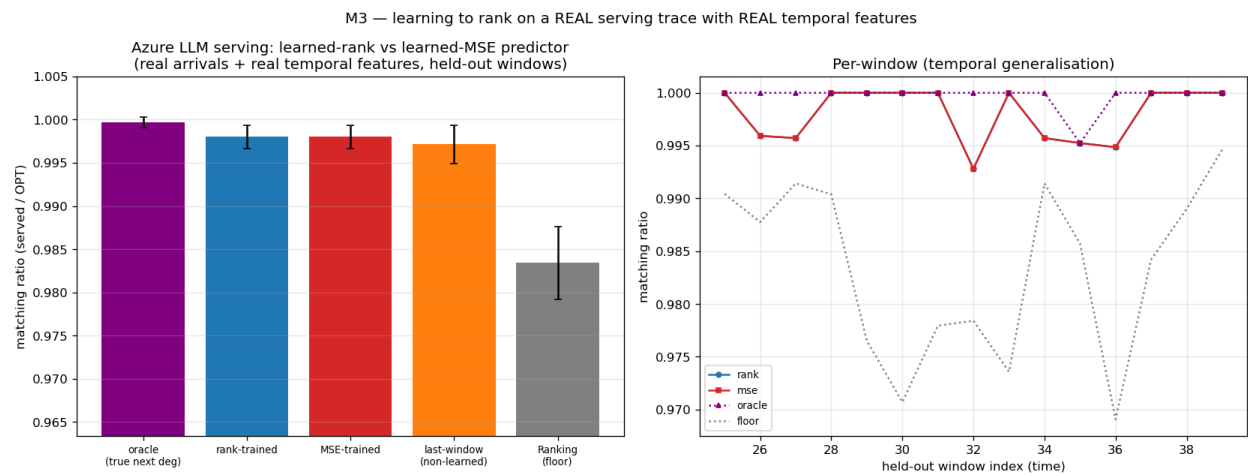


图 8.2: M3 (Azure trace, 真实时序特征): rank 训练与 MSE 训练的预测器不可区分; 驱动 rank 训练的人造发散并不出现。

果作为一个**强化核心发现**的负结果被并入本文：一旦预测器保序（廉价历史计数本已如此，第 7 章），在平均情况匹配上无论更好的算法还是更好训练的预测器都买不到多少。

8.2 用一个带预测的结果救援服务应用（一个负探针）

服务案例研究（第 9 章）复现了已确立的系统结论，因此被呈现为案例研究而非新意声明。我们追问是否有一个**新的**、可行动的带预测结果能救援它。若存在出路，必是一个反应式基线在其上确实远离最优的不同**目标**。除此之外的障碍又是那堵墙：我们所试的每个服务变体都优化吞吐（goodput），而吞吐是宽容的：过载下反应式路由填满容量的方式与最优相同。

我们探测了最有希望的候选：一个 **SLO/尾部目标**（在突发、非平稳负载下保护一类紧 SLO 请求不被丢弃），恰恰是反应式策略因缺乏前瞻可能失败的体制。用一个事件驱动模拟器，我们将非预测策略（静态容量预留；一个基于**观测**近期负载做预留的反应式自适应策略）与一个基于**实际未来突发**做预留的 **clairvoyant** 参照对比。这个设计有两点须先说明。该参照拥有完美前瞻，但并非该 SLO 目标下经证明的最优策略，因此它**估计**前瞻值多少钱，而非给出上界；而且这个估计明显不紧：在中度体制下，一个仅预留一个槽位的平凡静态预留就直接反超了它，这只能说明“按实际未来突发预留”在那里预留过头了。这次扫描真正确立的是单向的、因而仍然有用的结论：在所扫的每个体制（过载水平、均匀 vs 突发紧 SLO 需求）中，最佳非预测策略都落在（图 8.3）一个确切知道未来的策略的 $\leq 3\%$ 以内，所以这些策略欠缺的并不是预测所能提供的那种东西。两个原因，都对扫描稳健：保护一个紧 SLO 少数只需一点静态余量、无需预测；且突发足够持续，使得对观测负载做反应几乎和预测它一样好。

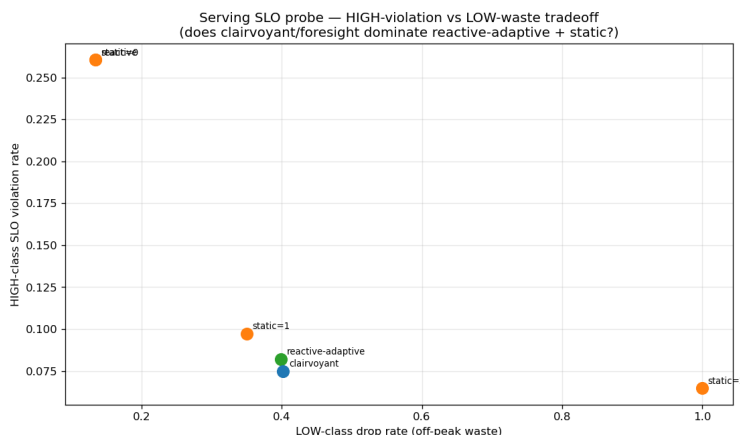


图 8.3: 服务 SLO 探针：非预测策略在几个百分点内匹配 clairvoyant 预言机，前瞻无济于事。

裁决。尾部目标也是宽容的：继吞吐（第 4–7 章）与预测器学习（8.1 节）之后，这是墙的第三张脸。我们没有找到前瞻有帮助的自然体制，故服务仍是案例研究。一个能打破这堵墙的体制（一个基线远离最优的非平稳或对抗目标）正是本文作为未来工作所框定的那类设置。

8.3 一次重定向了工作的文献综述

判定我们的哪些发现真正新颖，需要一次系统的 prior-art 综述而非直觉：我们从若干个独立的起点检索先行研究，并把每一条主张都回查原始文献而非其摘要。裁决有时令人沮丧。统一基准与对测试-回退的实证研究无人占据、值得领衔；顺序误差发现被 ACI 的附录 D 部分预占，须重构为紧度/度量刻画而非发现（第 5 章）；服务结论大多是已确立系统事实的重新推导，故应降格为案例研究（8.2 节、第 9 章）。第二次聚焦检查确认此前无工作量化前缀检验在强基线实例上的代价，同时标出任何此类结果都须防范的风险（Choo 等人构造性的基线耦合），这是 10.2 节展望的一项输入。

这次综述本身是此处所报告研究过程的一部分：它把一堆无差别的发现变成了有优先级的贡献，将精力从低价值的细化重定向到那个理论问题，并贯彻了全文的诚实护栏。

8.4 综合：负结果指向同一堵墙

三次探索指向同一方向。更好训练的预测器在真实特征上无帮助（8.1 节）；带预测的视角即便在尾部目标上也无法救援服务（8.2 节）；文献综述确认没有唾手可得的性能胜利（8.3 节）。连同第 4-7 章的吞吐墙，它们表明该现象不限于单一目标、单一算法或单一数据集：在我们所推的每个方向上都反复出现，而这种稳健性正是暗示这堵墙可能是被迫而非偶然的东西：这正是结论章展望（10.2 节）所接手的问题。

第九章 应用案例研究：AI 推理服务

本文的抽象并不局限于经典的匹配市场；它可实例化一个当代系统问题。本章将 AI 推理服务系统中的请求路由建模为在线匹配。本章做两件事。它检验这套抽象是否丰富到足以承载一个真实的系统问题。答案是它复现了该领域已确立的结论，这是能拿到的最强证据，说明这一映射是忠实的；同时它提供了第 8 章据以探测、并最终未能找到“预测真正有帮助的体制”的那个场景。由于下文复现的系统事实都是已知的，本章是一项**案例研究而非新意声明**；这一定位遵循第 8 章的先行研究综述。

9.1 服务实例化

我们将服务映射为在线**带容量匹配**：每个离线资源是一个模型副本或缓存分片，最多并发服务 c 个请求，因此我们做的是容量为 c 而非 1 的匹配（即所有 b 都等于 c 的 b -匹配；全文与图中一律写 c ）。到达是从一个非均匀、突发的流量分布中抽取的请求，一条边是一种能力或缓存亲和性。有两个量必须分清：原始 **goodput** 是被服务请求占到达请求的比例；而**竞争比**是被服务数除以同一实现实例上的带容量最优。二者只有在最优能服务全部到达时才相等，而在过载下它做不到，所以我们全程报告竞争比。图 9.1 与图 9.2 画的都是它；图 9.3 衡量的是另一个目标（KV 缓存命中率），并已在图中注明。

对图 9.2 的动态实验而言，那个最优不是一个匹配而是一个调度问题：从请求中挑一个子集接纳，每个在其整个服务时长内占用一个相容副本，每副本最多并发 c 个。它一般是 NP 难的，因此我们除以一个可计算的上界：把每副本容量松弛为每类型容量 $c \cdot \deg(\ell)$ ；松弛后的问题按类型分解为区间调度，可以精确求解。因此所报比值是真实竞争比的下界，而且这个界很紧：一个可行的离线分配把最优夹在到达数的 1.1% 以内（ $c = 3$ ）、0.4% 以内（ $c = 6$ ），在 $c = 12$ 时完全重合。我们在一个合成服务拓扑（图 9.1，那里可以直接扫描预测质量）与两条真实 trace 上考察这一实例化：一条 Azure LLM 推理 trace（图 9.2）与 Mooncake 前缀缓存 trace [20]（图 9.3）。第三条真实 trace（Wikipedia 页面浏览）用在 7.1 节的预测器研究中。

容量作为鲁棒性（图 9.1）。增大容量 c 会平滑坏预测的影响：一个容量感知的基线保持安全，而盲目信任一个流量预测则随容量增大而**进一步退化**。在我们所扫的容量范围内，增加容量买到的保护与自适应检验买到的相同：这是鲁棒性保险论点的系统对应物，也提醒我们二者中更便宜的那个是一个扩容决策，而不是一个算法决策。

预测 vs 实时负载（图 9.2）。在动态服务时间下（请求占据一个槽位一段真实时长、按事件逐一释放），

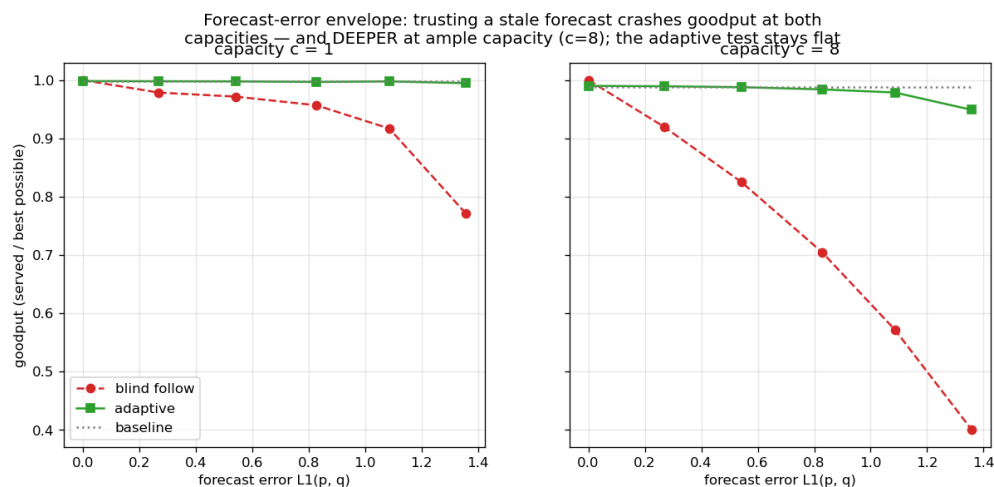


图 9.1: 容量作为鲁棒性（合成服务拓扑：200 个副本、40 个度数为 8 的请求类型、800 个到达、每点 25 次试验）：盲目跟随预测使竞争比崩塌，容量充裕（ $c=8$ ）时更深，而自适应检验保持平坦。

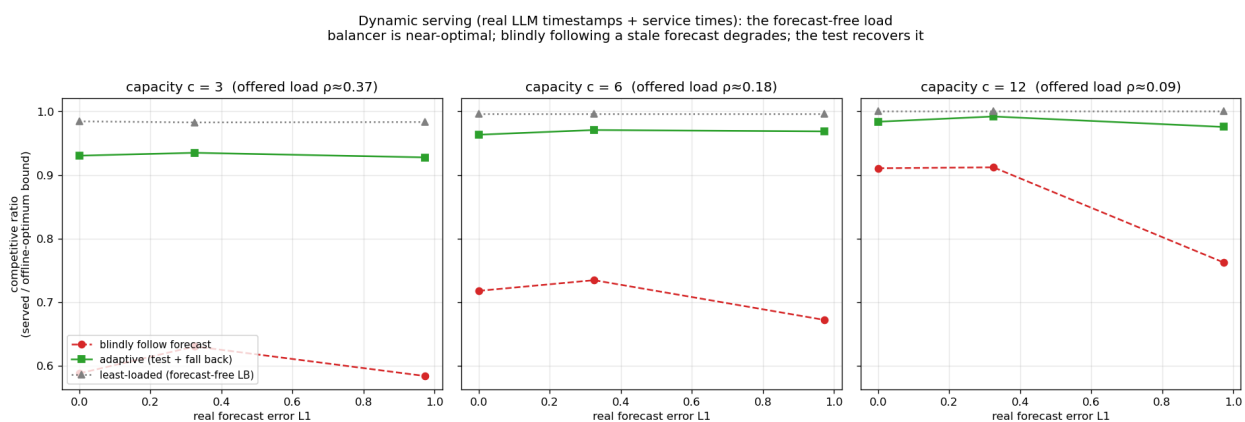


图 9.2: 动态服务时间下实时负载胜过陈旧预测（Azure LLM trace, 9683 个到达，每点 8 个拓扑）：免预测的均衡器在每个容量下都在离线最优的 2% 以内，盲目跟随最多损失 40 个百分点，而前缀检验挽回其中大部分。比值是相对离线最优的一个上界计算的，因而是保守的。

一个实时负载信号胜过一个陈旧的流量预测。相对离线最优来看，差距十分醒目：免预测的最少负载均衡器在每个容量下都达到最优的 0.98–1.00，而盲目跟随预测只有 0.58–0.91，前缀检验挽回其中大部分 (0.93–0.99)，代价是它所花掉的那段前缀。反应式策略不只是胜过跟随预测的那个；它已经落在完美事后诸葛所能达到的 2% 以内。

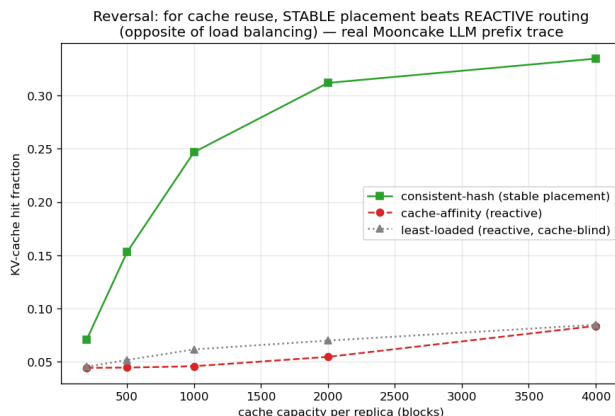


图 9.3: 缓存亲和反转 (Mooncake trace, 16 个副本, 缓存容量从 200 扫到 4000 个块): 对 KV 缓存复用, 稳定放置胜过反应式路由。纵轴是 KV 缓存命中率, 不是竞争比。

缓存亲和路由 (图 9.3)。对前缀缓存感知的路由, 一个稳定的亲和路由器胜过一个反应式的, 与流量预测的情形相反, 因为缓存局部性奖励持久性 [21]。

这些各自都干净地复现了一个已确立的系统结论, 表明该框架忠实地实例化了该问题。它们还合成一个观察: 前两条说的是**反应胜过预测**, 第三条说的正好相反, 而区别在于目标: 缓存局部性奖励持久性, 负载均衡奖励响应性。

9.2 一次寻找真正新结果的探针, 及其负结果

我们也追问了: 带预测的视角能否在尾部而非吞吐目标上给出一个**新的**、可行动的服务结果。答案是否定的: 在一个 SLO 目标 (在突发负载下保护一类紧 SLO 请求) 上, 一个非预测策略在所扫的每个体制中都落在一个拥有完美前瞻的策略的 $\leq 3\%$ 以内。该实验、它的两点保留与两条解释见 8.2 节。对本章而言, 推论是我们没有找到前瞻有帮助的自然体制, 故服务仍是案例研究。

9.3 本章小结

这套抽象触及了一个真实的系统问题并复现了其已确立的结论, 而且即便一个尾部目标也未开启一个真正的带预测胜利。剩下要问的是: 这几章反复撞上的这堵墙, 究竟是我们所选输入的偶然, 还是某种被迫: 这正是结论章接手的问题。

第十章 结论与未来工作

10.1 总结

本文研究了面向在线二部匹配的学习增强算法：先构建一个共同的实验基础，再解释它所揭示的东西。在同一套框架上（对照 Borodin 等人已发表的研究验证，见第 3 章），我们比较了免预测基线、MinPredictedDegree 及其增强、以及测试-回退算法，跨越合成图、六个真实世界图与真实请求 trace（第 4-7 章）。

在每一种设置上都浮现同一图景：在平均情况输入上免预测基线本已近乎最优，故一个好预测的一致性上升空间很小；无防护地跟随预测在坏预测下会崩到低于基线；而成熟算法的价值是下行保护，由结构式或自适应式鲁棒机制以不同取舍提供。

沿途锐化了两个子问题。那（微小的）损失由 Kendall- τ 顺序误差而非幅度支配，其中顺序依赖性本身是 Aamand、Chen 与 Indyk 的在先结果（第 5 章）。自适应检验有一个阈值校准病理与一个分辨率极限（第 6 章）。我们同时报告两个未奏效的方向：学习预测器以榨取更多性能，以及寻找一个预测真正有帮助的服务场景（第 8 章）。

这堵反复出现的墙引出一个显然的问题：它是我们所选输入的偶然，还是被迫的？下一节以展望的形式给出我们的回答。统一全文的一句话是：在平均情况的在线匹配上，预测是鲁棒性保险而非性能杠杆——而其上升空间小于“判定预测是否可信”所需付出的代价。

10.2 理论展望：检验建议的代价

本节证明一条不等式，然后据它解读本文自己的实验；除该不等式外，本节不把任何东西作为定理来声称。实验表明，没有任何实用的接受阈值能安全地捕获上升空间（6.3 节，图 6.4）；本节要问的是，某种别的决策规则是否能做到。

一个权衡不等式。 设 G 与 Bd 是共享同一建议的两个实例分布，跟随建议相对免预测基线，在 G 下获益 δ 、在 Bd 下受损 Δ ；记 γ_k 为二者长度为 k 的前缀分布之间的全变差距离，直观地说就是一个只看得到前 k 个到达的检验所能达到的最高准确度。则每个测试-回退算法，

无论其在前缀上用**任何**可测规则做决策，都满足

$$(1 - \eta_c) \leq \eta_r + \gamma_k + o(1),$$

其中 η_c 是它在 G 下放弃的上升空间比例， η_r 是它在 Bd 下以 Δ 为单位的鲁棒性损失。

证明是一个简短的条件化论证：捕获上升空间迫使算法在 G 下跟随，鲁棒性迫使它在 Bd 下回退，而前缀的任何函数都无法在两个统计上 γ_k -接近的前缀分布上表现不同。“既一致又鲁棒”于是变成一个**样本复杂度**问题：前缀要多长，才能把值得跟随的建议与应当拒绝的建议区分开？

在产生图 6.4 的稀缺资源实例上（一个**可分解族**，实例分裂为彼此独立的小格、建议在每个小格上分别起作用），答案取一条**预算-赌注**关系的形式。记 δ 为**赌注**：好建议相对基线的收益。在一条本文并未证明的预算定律下，做出决策需要 $k^* = \tilde{\Theta}(\theta/\delta^2)$ 的前缀，即赌注的平方反比，乘以一个与基线松弛 $1 - \rho_{\text{base}}$ 同阶的争用参数 θ ，且可由一个简单的**方向性**统计量达到（前缀到达与建议相符多、还是相悖多？）。赌注又被同一个松弛封顶：建议只能赢下基线留在桌面上的那部分。代入可得，任何低于 $\Theta(\sqrt{(1 - \rho_{\text{base}})/n})$ 的上升空间，都不会被任何规则在任何前缀长度 $k \leq n$ 下捕获。在第 4 章的参数 ($\rho_{\text{base}} \approx 0.99$ 、 $n = 2000$) 下该阈值 ≈ 0.004 ，恰是我们测得的上升空间的量级 (F3)。经验之墙正落在该解读所指的位置：潜在上升空间与可捕获上升空间随基线变强而分离，因为赌注缩小的速度快于检验分辨率改善的速度。

两条注记框定其范围。这条关系是双向的：赌注大（基线弱）之处，方向性统计量很便宜，跟随好建议是容易的，所以这堵墙是关于强基线、平均情况输入的陈述，而非关于检验本身；它并非“分布检验很贵”的后果，而是赌注太小的后果，6.3 节 ℓ_1 阈值的失明来自它检验的是**距离**而不是**收益**。此外，本文在此仅声称上面陈述的那条不等式：预算定律本身，以及它能否推广到可分解族之外，均未在本文中确立 (10.4 节、10.5 节)。

10.3 批判性评估

本项目源自一个选题要求——复现并扩展 Borodin、Karavasilis 与 Pankratov 的实验研究——并在动手前由我自己的开题提案作了重新定位。本节先对照这两者评估，再回头审视一路上做出的设计选择。

对照原始选题要求。选题要求了三件事。复现其中部分实验：做到了，而且是最先做的 (3.6 节)——所核对的五条定性论断全部复现，绝对差异小于 0.02，且是在不同的实现语言与不同的最大流例程之下。在不同类型的图上比较性能：做到了——两个合成图族，外加覆盖社交、生物与经济三类结构的六个真实世界图 (7.2 节)，两条承重结论在其上普遍成立。通过测试更多算法来扩展：做到了，但**主要不是选题建议的那个方向**。选题点名的是流式或离线算法；我的提案把它替换成了学习增强这一族，理由是该方向的理论已经跑到了实验前面，因此统一比较是更有价值的扩展。这一替换我在动手前与导师确认过，现在我仍认为是对的。附录 A.7 以缩减的规模补上了选题建议的那一支，并把它搭成一条松弛阶梯而非一组并列基准；它并非陪衬：在九个图中的七个上，多跑一遍数据带来的收益超过一个完美的度数预测，这就把残余差距定位在了不可撤销性上，而不是定位在信息不足上。

对照我自己的提案。提案设了三个研究问题，它们在本文中的落实并不均匀。*RQ1*：算法按其已证明的

保证排序，是否与它们在实践中排序一致——回答了，答案是否定的：为最坏情况设计的 Feldman 与 Jaillet-Lu 在平均情况输入上是最弱的免预测项（0.760 与 0.788，低于 Greedy 的 0.890；在经济图上是 0.73–0.77），却同时高出自己的最坏情况界 0.03–0.06。这正是 Borodin 等人母问题的原样继承，并在带预测的设定下作答。*RQ2*：性能-预测误差曲线的形状，以及不同算法的曲线在何处交叉——只回答了一部分。第 5 章在一个轴上给出的比我原先要的更多——相对顺序误差的那条曲线，四种误差模型都塌缩其上——但第 4 章报告的是离散的质量列而非连续曲线，因此“曲线在何处交叉”只通过一致性-鲁棒性平面作了定性回答，并未定位交点。*RQ3*：测试-回退机制的实际代价——三者中回答得最完整：检验代价、在已发表阈值与重校准阈值下的误判率，以及不可撤销惩罚（第 6 章）。

答应了却没有交付的。有一处缺口是实质性的。提案 6.2 节把到达顺序作为独立自变量，要求对随机序、对抗序与真实时间序分别报告，理由（正确的理由）是随机到达与对抗到达是一套不同模型。而本文全程在 known-i.i.d. 模型下工作。真实时间序确实出现了，通过第 7 章与第 9 章的 Wikipedia 与 Azure trace，但没有对抗序这一支。这个遗漏比在提案时看起来更要紧，因为核心发现恰恰是“免预测基线已接近最优”——而 known-i.i.d. 正是这一点最成立的模型。砍掉对抗序这一支，等于移走了唯一一个可能证伪我自己主结论的条件；这也正是它排在 10.5 节未来工作首位的原因，而不是作为一个顺理成章的延伸。第二处较小：提案把运行时间列为辅助指标，而本文只报告了一对墙钟数字（7.1 节），没有系统的计时。

事后看这些选择。有三项是对的。沿实例的结构注入误差而非注入独立同分布噪声，本就是提案自己的设计赌注（提案第 5 节称之为核心方法学创新），而且如愿兑现：独立同分布噪声会抹平幅度与顺序的区分，第 5 章向 Kendall- τ 的坍塌将完全不可见。在框架上构建任何东西之前先对照 Borodin 等人做验证，花了约一周，换来的是把日后每一处差异都归因于算法而非基础设施的资格。而把理论主张收缩到真正证明了的，那一条不等式，正是 Q3 的答案得以站得住的原因。

除上面的到达序缺口外，还有两项可商榷。只以匹配规模为目标，是最限制结论适用范围的选择：这堵墙是关于吞吐的陈述，而唯一一次针对尾部目标的探针（8.2 节）来得晚，且只关闭了最简单的一种尝试。以及，Python 实现把实例规模限制在 $n \approx 2000$ ；为可复现性这是划算的交换，但 10.2 节的解读预测可捕获阈值按 $1/\sqrt{n}$ 变化，更大的实例本可以在该预测最锐利的地方检验它。

有一条流程教训值得记下。学习排序的探索（8.1 节）先跑了合成的 M0 与 M1，才跑 M3——即在真实时序特征上的检验；而 M3 既是更便宜的实验，也是给出否定结论的那一个。先跑贴近现实的检验，本可以用一小部分工作量得到同样的结论。

总体。选题要求达成，代价是一处我愿意为之辩护、并已在缩减规模上补做的替换；我自己的三个研究问题中两个完全达成、第三个部分达成；而提案中的一项承诺——对抗到达序那一支——未能交付。实验部分是本文扎实的那一半，理论部分则是有意收窄的。最不可能被推翻的结论是墙本身，它在我改变过的每一种算法、误差模型、图族与预测器上都反复出现；而诚实的另一面是：最有可能推翻它的那个维度，恰恰是我没有改变的那个。

10.4 局限

- **输入模型。**我们在 known-i.i.d. 模型下工作。由于每个 known-i.i.d. 实例也是一个随机序实例，在随机序模型中证明的保证可以沿用到我们这里（但反之不然，见 2.1 节）；不过经验之墙是一个平均情况陈述，我们不为对抗到达序声称它。
- **检验模型。**遵循原作者，测试-回退实验使用一个经验 ℓ_1 代理来替代（未实现的）分布检验器；10.2 节解释了该代理的失明是结构性的，而非实现产物。
- **预测对象异质性。**度数预测族与直方图预测族并不映射到每个图上，故我们在并列面板而非同一张表中报告。
- **数据广度。**每种真实模态由一个 trace 检验。
- **目标函数。**我们只以匹配规模（goodput）衡量。在免预测基线并不近乎最优的目标上（尾部延迟、逐类型公平、迁移或重算成本），图景可能不同；我们在那个方向上的唯一一次探测（8.2 节）关闭了最简单的尝试，但未关闭整个空间（10.5 节）。
- **理论范围。**本文有意将其理论限定于 10.2 节的展望：一条已证明的权衡不等式加上对实验的定量解读。完整的预算-赌注定律是筹备中的后续工作；除该不等式外，本文不声称任何定理。

10.5 未来工作

本文框定而非解决了预测可能真正帮助在线匹配的那些设置，它们是自然的下一步方向。

- **超越平均情况输入。**这堵墙是平均情况现象。对抗或非平稳到达序（其中免预测基线可证远离最优）正是预测应携带真实价值之处，也是本文这堵墙的一个正向对应结果可能被证明之处。
- **超越吞吐。**基线并不近乎最优的目标（尾部延迟、逐类型公平、迁移或重算成本）或许允许 goodput 目标所排除的真正带预测收益；我们的服务 SLO 探针（第 8 章）关闭了最简单的此类尝试，但未关闭整个空间。
- **完成理论。**完成后续的预算-赌注发展，即锋利的双侧定律、作为其匹配上界的方向性统计量、及其确切范围（可分解族；非可分解族能否推高预算是开放的），将把 10.2 节的展望变成一个紧刻画。
- **完整的流式与离线那一支。**附录 A.7 的阶梯是刻意做小的。补全它——内存核算、更强的 $(1 - \varepsilon)$ 多逼近算法、对抗性的边到达序——可以检验当流式一侧换成强算法而非最简单的算法时，“修改胜过建议”这一读法是否依然成立。
- **诚实地寻找更好的检验。**一个超线性或摊还的检验，或一个复用在线决策作为样本的检验，能否胜过 10.2 节的赌注平方预算，是一个开放且有实际动机的问题。

10.6 结语

预测是在线算法的一个强大工具，但本文是对其在一个被充分理解的问题上局限的研究。在平均情况的在线匹配上，诚实的裁决有三部分。一个廉价、保序的预测器本已捕获了几乎全部可捕获的东西。成

熟的机器以保险而非性能的方式发挥价值。而在这些输入上，判定一个预测是否可信的代价超过了该预测本身的价值。认识预测在何处无能为力，我们希望，与知道它在何处有用同样有价值。

参考文献

- [1] R. M. Karp, U. V. Vazirani, and V. V. Vazirani. An optimal algorithm for on-line bipartite matching. *STOC*, 1990.
- [2] J. Feldman, A. Mehta, V. Mirrokni, and S. Muthukrishnan. Online stochastic matching: Beating $1-1/e$. *FOCS*, 2009.
- [3] V. H. Manshadi, S. Oveis Gharan, and A. Saberi. Online stochastic matching: Online actions based on offline statistics. *Mathematics of Operations Research*, 37 (4), pp. 559–573, 2012.
- [4] P. Jaillet and X. Lu. Online stochastic matching: New algorithms with better bounds. *Mathematics of Operations Research*, 39 (3), pp. 624–646, 2014.
- [5] A. Borodin, C. Karavasilis, and D. Pankratov. An experimental study of algorithms for online bipartite matching. *ACM Journal of Experimental Algorithmics*, 25, 2020.
- [6] M. Mitzenmacher and S. Vassilvitskii. Algorithms with predictions. *Beyond the worst-case analysis of algorithms*, Cambridge University Press, pp. 646–662, 2020.
- [7] T. Lykouris and S. Vassilvitskii. Competitive caching with machine learned advice. *ICML*, 2018.
- [8] A. Wei and F. Zhang. Optimal robustness-consistency trade-offs for learning-augmented online algorithms. *NeurIPS*, 2020.
- [9] T. Yoshinaga and Y. Kawase. Analyzing the effect of prediction accuracy on the distributionally-robust competitive ratio. *arXiv preprint arXiv:2601.06813*, 2026.
- [10] J. Chłędowski, A. Polak, B. Szabucki, and K. Żoła. Robust learning-augmented caching: An experimental study. *ICML*, 2021.
- [11] A. Aamand, J. Y. Chen, and P. Indyk. (Optimal) online bipartite matching with degree information. *NeurIPS*, 2022.
- [12] D. Choo, T. Gouleakis, C. K. Ling, and A. Bhattacharyya. Online bipartite matching with imperfect advice. *Proceedings of the 41st international conference on machine learning (ICML)*, 2024.
- [13] J. Jiao, Y. Han, and T. Weissman. Minimax estimation of the L_1 distance. *IEEE Transactions on Information Theory*, 64 (10), pp. 6672–6706, 2018.
- [14] K. Burathep, T. Erlebach, and W. K. Moses Jr. Learning-augmented online bipartite matching in the random arrival order model. *Proceedings of the 51st international conference on current trends in theory and practice of computer science (SOFSEM)*, 2026.

- [15] L. Paninski. A coincidence-based test for uniformity given very sparsely sampled discrete data. *IEEE Transactions on Information Theory*, 54 (10), pp. 4750–4755, 2008.
- [16] G. Valiant and P. Valiant. An automatic inequality prover and instance optimal identity testing. *SIAM Journal on Computing*, 46 (1), pp. 429–455, 2017.
- [17] G. Valiant and P. Valiant. Estimating the unseen: An $n/\log n$ -sample estimator for entropy and support size, shown optimal via new CLTs. *STOC*, 2011.
- [18] C. L. Canonne, A. Jain, G. Kamath, and J. Li. The price of tolerance in distribution testing. *COLT*, 2022.
- [19] J. E. Hopcroft and R. M. Karp. An $n^{5/2}$ algorithm for maximum matchings in bipartite graphs. *SIAM Journal on Computing*, 1973.
- [20] R. Qin, Z. Li, W. He, M. Zhang, Y. Wu, W. Zheng, and X. Xu. Mooncake: A KVCache-centric disaggregated architecture for LLM serving. *arXiv preprint arXiv:2407.00079*, 2024.
- [21] V. Srivatsa, Z. He, R. Abhyankar, D. Li, and Y. Zhang. Preble: Efficient distributed prompt scheduling for LLM serving. *arXiv preprint arXiv:2407.00023*, 2024.
- [22] J. Feigenbaum, S. Kannan, A. McGregor, S. Suri, and J. Zhang. On graph problems in a semi-streaming model. *Theoretical Computer Science*, 348 (2–3), pp. 207–216, 2005.
- [23] A. McGregor. Finding graph matchings in data streams. *APPROX-RANDOM*, pp. 170–181, 2005.

附录 A 复现指南

本文中每个定量结果都由单个脚本从固定种子重生成。这些脚本须分为两类。多数是**自足的**：它们自行生成合成实例，在一份干净的代码检出上开箱即跑。其余的（第 7、8、9 章的真实图与真实 trace 结果）需要先把外部数据放到 `data/` 下；这些数据不随代码分发，A.5 节记录了每个数据集是什么、从哪里来。下面的映射表中以匕首号 (†) 标出第二类脚本。

本附录将每张图与表映射到其脚本，给出命令与运行时间，列出第 3 章推迟至此的完整 Phase-2 复现表，并记录数据来源。

A.1 环境与原则

- **技术栈**：Python 3.12；NumPy 1.26+、SciPy 1.13、NetworkX 3.3、Matplotlib（仅绘图）。
- **确定性**：所有随机性都经 NumPy 的 `default_rng(seed).spawn(k)` 从单一主种子（默认 0）派生，得到独立、可复现的流（图、实例、算法随机性、预测扰动）。结果在 NumPy 1.25 起的小版本间按位稳定（基于 BitGenerator 的 `spawn`）。
- **配对试验**：在一次比较中，每个算法与误差水平复用同一图、到达序列、OPT 与平局种子，故差异可归因于预测本身。
- **置信区间**：试验上的 95% 正态近似半宽。
- **流分解**。有三处预处理（Feldman、Jaillet-Lu 与服务端的建议 b -匹配）取一个最大流并消费它的分解；与流的值不同，分解并不唯一。它们的网络用整数而非字符串标注节点，使 NetworkX 返回的分解在多次运行间稳定；用字符串标注时，它会随 Python 每进程的字符串哈希而变，由此派生的每个数字都会在同一脚本的两次运行之间发生变化。
- 每个脚本写出 `results/<name>.json`（原始均值/置信区间）与 `results/<name>.png`（图），并打印逐步进度。

A.2 图/表 → 脚本映射

论文对象	脚本	输出	约运行时间
图 3.1 (ER U 曲线)	scripts/run_er_full.py	results/er_full.{json,png}	20 分钟
图 3.2 (左正则)	scripts/run_left_regular.py	results/left_regular.{json,png}	10 分钟
3.1 / 3.5 节方法学核对	scripts/run_metric_checks.py	results/metric_check.json	90 秒
表 4.1 (统一基准)	scripts/run_unified_benchmark.py 再	results/unified_benchmark.{json,png}、 unified_benchmark_panel{A,B,C}.png、 plot_unified_panels.py unified_benchmark_tables.md	10 秒
图 4.1 (一致性-鲁棒性平面)	scripts/run_consistency_robustness.py	results/consistency_robustness.{json,png}	1 秒
图 5.1 (顺序误差 vs ACI)	scripts/run_order_vs_theory.py	results/order_vs_theory.{json,png}	30 秒
图 6.1 (包络)、图 6.2 (前缀扫描)	scripts/run_choo_bem.py	results/choo_bem_{envelope,prefix}.png	120 分钟
图 6.3、重校准 (6.3 节)	scripts/run_recalibration.py	results/recalibration_*.png	1.5 分钟
图 6.4 (检验之墙边界)	scripts/run_impossibilities.py	results/impossibility_frontier.{json,png}	6 秒
图 7.1 (真实预测器)†	scripts/run_real_predictor.py	results/real_predictor.{json,png}	15 秒
图 7.2 (六个真实图)†	scripts/run_realworld_results/realworld_robustness.py	results/realworld_robustness.{json,png}	6.5 秒
图 8.1 (M0 rank vs MSE)	scripts/run_rank_vs_mse_mvel.py	results/rank_vs_mse_mvel.{json,png}	1 秒
M1 扫描 (8.1 节, 无图)	scripts/run_rank_when_it_matters.py	results/rank_when_it_matters.{json,png}	20 秒
图 8.2 (M3 真实 trace 学习)†	scripts/run_rank_real_trace.py	results/rank_real_trace.{json,png}	1 秒
图 8.3 (服务 SLO 探针)	scripts/run_serving_slo_probe.py	results/serving_slo_probe.{json,png}	1 秒
图 9.1-9.3、服务 (第 9 章)†	scripts/run_serving*.py run_prefix_cache.py	results/serving_*.png、 prefix_cache_*.png 等	不一
松弛阶梯、流式 (A.7 节)	scripts/run_streaming_ladders.py 再	results/streaming_ladder.{json,png}、 streaming_ladder_tables.md plot_streaming_ladder.py	40 秒
10.2 节展望核对 (A.6 节)	scripts/verify_witness_gap.py	控制台输出	数秒
真实图 Borodin 表 3/4 (验证)†	scripts/run_realworld.py	results/realworld.json	数分钟

运行时间为单机墙钟 (Apple M4 Pro, 12 核); 除 NumPy 自身的向量化外脚本为单线程, 因此随单核速度伸缩。

区间口径 (自 3.5 节移来)。run_metric_check.py 还把本文报告的按算法区间与同一批试验上的配对差区间做了对比。逐实例比值正相关处, 配对区间窄 1.7-2.6 倍 (相关系数 0.67 与 0.85); 不正相关处

(垃圾建议下的盲目跟随对 Ranking, 相关系数 -0.15), 配对并不占便宜, 按算法各自报告的区间才是诚实的那个。本文余量最小的一处, Feldman(MPD) 的 $+0.0044$ 对 ± 0.0023 , 配对后收紧到 ± 0.0009 。

A.3 复现命令

```
# 从项目根目录 (matching-experiments/)
# 正确性锚点: 8 个可手工验证的测试文件, 全部通过:
for t in tests/test_*.py; do python3 "$t"; done

# 重生成主要结果 (快脚本):
python3 scripts/run_unified_benchmark.py      # 表 4.1 (数据)
python3 scripts/plot_unified_panels.py        # 表 4.1 (面板小图; 需先跑上一行)
python3 scripts/run_consistency_robustness.py # 图 4.1 (需先跑 run_unified_benchmark)
python3 scripts/run_metric_check.py           # 3.1 与 3.5 节的方法学核对
python3 scripts/run_streaming_ladder.py       # A.7 节阶梯 (数据)
python3 scripts/plot_streaming_ladder.py      # A.7 节图 (需先跑上一行)
python3 scripts/run_order_vs_theory.py        # 图 5.1
python3 scripts/run_real_predictor.py         # 图 7.1
python3 scripts/run_realworld_robustness.py   # 图 7.2
python3 scripts/run_rank_real_trace.py        # 图 8.2
python3 scripts/run_serving_slo_probe.py      # 图 8.3
python3 scripts/run_impossibility_frontier.py # 图 6.4

# 较长 (受最大流 / Hopcroft-Karp 限制) 的扫描:
python3 scripts/run_er_full.py                # 图 3.1 (~20 分钟)
python3 scripts/run_left_regular.py           # 图 3.2 (~10 分钟)
python3 scripts/run_choo_bem.py               # 图 6.1、6.2 (~20 分钟)
```

除特别说明外所有脚本硬编码种子 0; 重跑即复现所报告的数字。

A.4 完整 Phase-2 复现表 (自 3.6 节推迟)

设定: $n = 1000$ 、 $m = n$ 、每参数值 100 次试验、种子 0。目标是与 Borodin 等人定性一致 (Python/NetworkX 对该文 C++/Edmonds-Karp; 接受绝对差 ≤ 0.02 , 故比值一律给到三位小数)。

Erdős-Rényi, 选定 c 处的竞争比 (SG=SimpleGreedy, Rk=Ranking, F/J=Feldman/Jaillet-Lu, -NG/-G= 非贪婪/贪婪):

c	SG	Rk	F-NG	F-G	J-NG	J-G
0.10	1.000	0.999	1.000	1.000	0.999	1.000
1.90	0.936	0.936	0.904	0.964	0.920	0.961
4.90	0.864	0.866	0.764	0.884	0.795	0.886
8.90	0.909	0.909	0.730	0.912	0.765	0.913
14.90	0.949	0.949	0.730	0.949	0.760	0.948

各算法最小值 (ER): SG 0.864 @ $c=4.9$; Rk 0.865 @ 4.7; F-NG 0.728 @ 14.5; F-G 0.884 @ 5.3; J-NG 0.759 @ 13.9; J-G 0.884 @ 5.3。

随机左正则, 选定 d 处的竞争比:

d	SG	Rk	F-NG	F-G	J-NG	J-G
1	1.000	1.000	0.985	1.000	0.985	1.000
2	0.954	0.954	0.877	0.968	0.895	0.966
5	0.890	0.890	0.758	0.900	0.788	0.901
10	0.928	0.928	0.733	0.928	0.766	0.928
30	0.977	0.977	0.730	0.976	0.760	0.976

论断核对清单 (全部验证 ✓): 贪婪最小值在 $c \approx 4.9 / d = 5$ 附近; Ranking $\approx$ SimpleGreedy (ER 最大差 0.0017、LR 0.0013, 该文正因此省略 Ranking 曲线); 非贪婪变体随 c, d 增大单调退化; 贪婪复杂变体渐近 $\approx$ SimpleGreedy; $c=14.9$ 处非贪婪的排序 (J-NG 0.760 > F-NG 0.730) 与该文最坏情况界的排序一致。跨族来看, 非贪婪算法收敛到相同的渐近常数 (Feldman 0.730、Jaillet-Lu 0.760, 相差在 0.001 内), 高于其最坏情况界 +0.06 / +0.03; 3.6 节给出对这一观察的解读。

A.5 数据来源

真实数据本地存放于 `data/` 下 (体量大; 不纳入版本控制)。

- **真实图 (第 7 章、3.6 节验证)**: 六个取自 Network Repository (networkrepository.com) 的图: `socfb-Caltech36`、`socfb-Reed98`、`bio-CE-GN`、`bio-CE-PG`、`econ-beause`、`econ-mbeaflw`, 为 MatrixMarket `.mtx` / 空白分隔 `.edges`, 化简为简单无向图, 并经随机平衡划分 (Borodin 表 3) 或复制双重覆盖 (表 4) 转为二部图。
- **Trace (第 7、8、9 章)**: 四天的 Wikipedia “每日热门文章”, 取自 Wikimedia REST 页面浏览 API (`data/trace/wiki/`, 用作直播日 vs 1/7/30 天陈旧的预测); 取自微软公开 Azure 数据集发布的 Azure LLM 推理 trace (`data/trace/azure_llm/`, 带时间戳的上下文/生成 token 计数); 以及随 [20] 发布的 Mooncake 会话 trace (`data/trace/mooncake/`, 每请求的前缀缓存块

hash_ids)。Wikipedia trace 是一个活数据源的快照而非静态基准，因此精确复现需要同一份快照，而不仅仅是同一个 API。

A.6 展望验证片段 (10.2 节)

10.2 节展望中引用的每一个量，在被使用之前都经过数值核对。共有三项核对，每项都是对稀缺资源构造的一次简短模拟。

1. **单 cell 常数。**每 cell 的最优、每 cell 的基线，以及优势与 ℓ_1 的闭式表达，与模拟吻合到三位小数。例如在争用率 $\theta = 0.6$ 、建议偏置 0.3 时，模拟得每 cell 优势为 ± 0.119 ，而预测值为 $\pm \theta \cdot 0.3 = \pm 0.12$ 。
2. **转换律。**跟随建议时的期望比值与 $\rho_{\text{perfect}} - \frac{1}{2}\ell_1(p, q)$ 吻合（其中 ρ_{perfect} 是完美建议下的比值），即它随建议的 ℓ_1 误差线性下降，同样精确到三位小数。用 10.2 节的语言说，正是这一点使赌注 δ 成为建议误差的线性函数。
3. **预算-赌注的两个成分。**方向性统计量在固定前缀长度下的准确率不随 n 增大而退化；而插入式 ℓ_1 估计在 $k \ll r$ 时变得失明，即无法区分好建议与坏建议。二者都由 `scripts/verify_witness_gap.py` 产生 (A.2 节)。

这些核对使 10.2 节的解读成立，但它们不是对它的证明。形式化发展是本文之外的独立工作。

A.7 松弛阶梯：流式与离线算法 (10.3 节)

原始选题要求建议用流式或离线算法来扩展复现工作。10.3 节记录了我改用学习增强这一族作为替换。本节以缩减的规模补做选题建议的那一支，并且不把它摆成一组并列基准，而是让它对本文自己的问题说话：把流式算法放在同一批实例的一条松弛阶梯上，从而衡量在线模型的两条约束中，究竟哪一条是承重的。

两个模型。正文工作在在线点到达模型下：一个在线点到达，暴露其全部邻居，必须立即且不可撤销地被匹配。内存不受限，但不允许修改。半流式模型 [22] 把两者反过来：图以边流的形式按任意顺序到达，工作内存为 $O(n \text{ polylog } n)$ ，即够放下一个匹配而放不下整张图，且算法可以扫 p 遍，因此当 $p > 1$ 时先前的决策可以被修改。按其允许的能力排列：

模型	内存	可修改?	能看到整个边集?
离线最优	不受限	是	是
流式, p 遍	$O(n)$	每多一遍可改一次	是, 每次一遍
流式, 1 遍	$O(n)$	否	是, 仅一次
在线点到达	不受限	否	否

两个算法。第一遍是贪心：只要一条边的两端都还空闲就取下它。结果是一个极大匹配，因而至少为 $\text{OPT}/2$ [22]。之后每一遍都是标准的长度为 3 的增广遍 [23]，在二部情形下特化为：扫描过程中为每

个已匹配的离线点记住一个空闲在线邻居、为每个已匹配的在线点记住一个空闲离线邻居（合计 $O(n)$ 个字），扫完后再提交它们所编码的一组点不相交的增广。离线那一级不需要新代码：Hopcroft-Karp 最优本来就是本文每一个比值的分母。选题里的离线建议也不需要单列一项：均匀随机边序下的一遍，恰好就是在随机置换的边表上跑离线贪心。

设置。与表 4.1 相同的三个合成图族、与 7.2 节相同的六个真实图（随机划分转换），相同的实例、相同的 Hopcroft-Karp 最优、相同的 95% 区间约定。为验证这条阶梯与正文站在同一基础上，它的在线各行在区间内复现了表 4.1: `clvb_zipf` 上 Greedy 为 0.916 对 0.917, `left_regular` 上为 0.889 对 0.890；完美预测下的 MPD 为 0.989 对 0.989、0.930 对 0.932；Ranking 在三个面板上三位小数完全一致。完整表格由脚本生成于 `results/streaming_ladder_tables.md`；下图绘出其中四行。

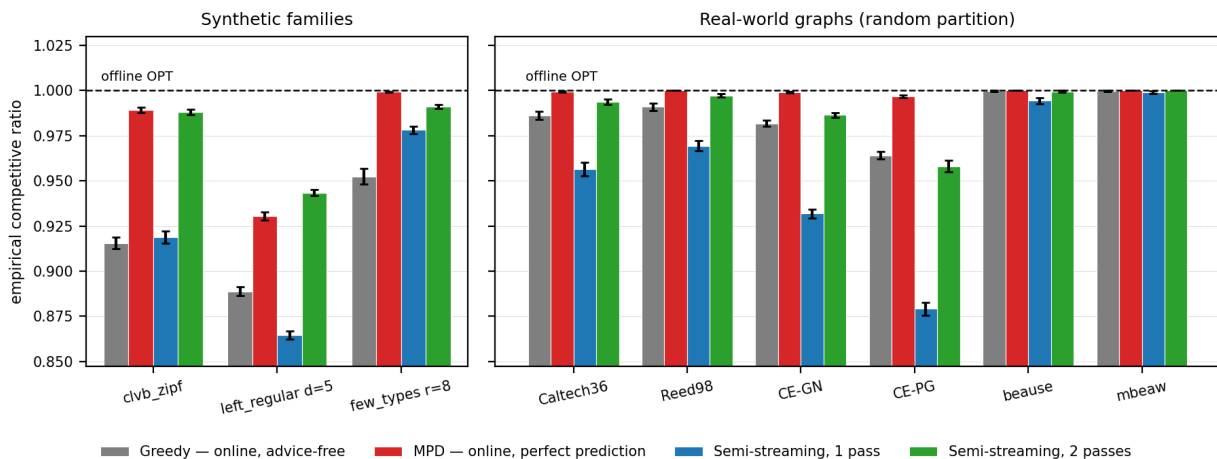


图 A.1: 松弛阶梯。灰与红是在线模型（免预测的 Greedy，以及给定完美度数预测的 MPD）；蓝与绿是一遍与两遍的半流式。在九个图中的七个上，多扫一遍带来的收益超过一个完美预测。

读法一：一遍并不是免费的午餐。在放宽内存但仍只扫一遍的情况下，结果在九个图中的七个上劣于在线模型：CE-PG 上是 0.879 对 Greedy 的 0.964，CE-GN 上是 0.932 对 0.982，`left_regular` 上是 0.865 对 0.889。两者产出的都是极大匹配，都在 $OPT/2$ 之上；差别在于哪一个极大匹配。点到达把一个点的边聚在一起，因此该点只要在到达时还有空闲邻居就会被匹配；随机边序把它们打散，于是一个离线点可能被一个本来还有其他选择的在线点占走。例外是 `few_types`，那里一遍达到 0.978 而 Greedy 只有 0.952，原因与 Ranking 在那里也胜过 Greedy (0.990) 相同：Greedy 固定的字典序打破平局把到达集中到低下标的离线点上，而随机边序等于把这个平局随机化了。

读法二：第二遍才是杠杆。一次增广遍把每个图都抬升到距离线最优 0.6%–4.2% 之内。对等的比较是边际比较：每个模型都有一个既不预测也不修改的条目（在线的 Greedy、流式的一遍），于是可以问在它之上加一项能力各买到多少。下表第一列是在线模型相对 Greedy 的增益，另两列是流式模型相对它自己那一遍的增益。

图	+ 完美预测	+ 1 次修改遍	+ 2 次修改遍
clvb_zipf	+0.074	+0.069	+0.072

图	+ 完美预测	+ 1 次修改遍	+ 2 次修改遍
<code>left_regular</code>	+0.042	+0.079	+0.087
<code>few_types</code>	+0.047	+0.013	+0.017
Caltech36	+0.013	+0.037	+0.041
Reed98	+0.009	+0.028	+0.029
CE-GN	+0.017	+0.055	+0.063
CE-PG	+0.033	+0.079	+0.099
beause	+0.000	+0.005	+0.006
mbeaw	+0.000	+0.001	+0.001

在九个图中的七个上（包括全部六个真实图），一次修改遍的价值高于一个完美的度数预测。两个例外是 `clvb_zipf`（两者相差在 0.005 以内）与 `few_types`，后者正是被刻意构造造成近乎完美可匹配、且类型计数成块的那个图族，也就是建议类算法本就为之设计的那一个场景。

读法三：这对本文的主结论说明了什么。正文的发现是，预测是鲁棒性保险而非性能杠杆，因为免预测基线本就已接近最优。这条阶梯把残余差距定了位：它不是一个更好的预测器能填上的信息缺口，而是第 6 章直接量化过其代价的不可撤销性约束。对同一批数据多看一眼——没有预测器、没有训练集、没有建议，而且内存比在线算法被允许的还少——所收回的差距，比完美的度数信息还多，唯一的例外是那个为度数信息而生的图族。

范围，以及这不是什么。第二遍并不是在线算法。它要求到达仍然可取、决策尚未被执行，而这恰恰是本文所面向的那些应用所禁止的：已派出的司机、已投放的广告位收不回来。在那种场合，两条路线中只有预测这一条可用，本节的阶梯并不构成对它的反驳。阶梯衡量的是两条路线各自上行空间的大小，而不是它们的适用性；其结论是，带预测文献在这一区间所追逐的上行空间，并不比一次修改所买到的更大。这同样是选题建议的一个缩减版本：只有一个流式算法族、最多三遍，没有内存核算，没有与更强的 $(1 - \varepsilon)$ 近似流式算法比较，边序也是均匀随机而非对抗性的。10.5 节说明了完整的一支还需要什么。